



Scheduling two interfering job sets on identical parallel machines with makespan and total completion time minimization

Tifenn Rault¹ · Faiza Sadi¹ · Jean-Charles Billaut¹ · Ameer Soukhal¹

Accepted: 10 June 2024 / Published online: 23 July 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

We consider a two-agent scheduling problem with interfering job sets. Agent *A*—which can be considered as the resource manager—is associated with the whole set of jobs, and agent *B*—which can be considered as an application master—is associated with a subset of jobs. Each agent aims at minimizing either the maximum or the total completion time of its jobs. Considering an identical parallel machines environment, the goal is to find an assignment and a schedule of jobs which represents the best compromise between the requirements of the agents. The class of multi-agent scheduling problems has drawn a significant interest to researchers in the area of scheduling and operational research. When both agents minimize the makespan, we prove that the number of Pareto solutions is bounded and we show that this bound is reached. Using the ε -constraint approach, we propose two integer programming formulations that allow to obtain the exact Pareto front for each problem. Given that the studied problems are NP-hard, we propose genetic algorithms (NSGA-II) to generate approximated Pareto fronts. Computational experiments are conducted to analyze the performances of the proposed methods. The results indicate that the genetic algorithms provide high-quality Pareto fronts and are computationally efficient.

Keywords Multi-agent scheduling · Parallel machines · Pareto front · Genetic algorithms · Integer programming

1 Introduction

Most approaches to tackle multicriteria scheduling problems assume that there is only one decision maker and that the problem is to find the best compromise solutions (T'kindt and Billaut 2006). However, it may happen that the measure characterized for some jobs is not appropriate for other jobs. For example, some jobs in a given workshop may have hard due dates and others may have flexible due dates. For the jobs with hard due dates, the problem is to schedule these jobs so that they are completed on time (no tardiness allowed), and for the others, the problem is to minimize a tardiness-based

criterion. For such scheduling problems, the multi-criterion models are not appropriate (Agnetis et al. 2014).

Within this context, *multi-agent scheduling problems* consider that several agents are competing for the use of the same resources. Each agent is responsible for a set of jobs and aims at minimizing one objective function that depends only on the completion times of its associated subset of jobs. When the subsets of jobs are disjoint, the problems are called *competing job set problems* (Agnetis et al. 2000; Baker and Smith 2003; Agnetis et al. 2004). In this paper, we are interested in a specific class of two-agent scheduling problems that is known in the literature as *interfering job sets with a global agent*. These scheduling problems are presented in Huynh Tuong et al. (2012) in a single-machine environment. The authors consider several agents having disjoint subsets of jobs and their own objective functions. An additional objective function concerning the entire set of jobs is also considered. This function can be considered as a dummy “global” agent objective function, often considered to be the owner of the production system. In this case, its purpose is to ensure the optimization of the global system with its own objective function.

This type of problems may be found in many manufacturing and computing systems where the aim is to optimize

✉ Tifenn Rault
tifenn.rault@univ-tours.fr

✉ Jean-Charles Billaut
jean-charles.billaut@univ-tours.fr

✉ Ameer Soukhal
ameur.soukhal@univ-tours.fr

¹ LIFAT, ROOT ERL-CNRS 7002, Université François Rabelais de Tours, CNRS, 64 Avenue Jean Portalis, 37200 Tours, France

the scheduling of the system resources, even if several intern agents may have other performance criteria. As an illustrative example, for scheduling problems in grids, jobs are assigned and executed on shared computing clusters to spread the load. Each set of tasks is submitted by a user. The system manager (global agent) must then honor its commitment to each user (agent) with its concern for load balancing. In Cordeiro et al. (2011), the authors consider that each cluster is associated with an agent. The criterion for each cluster is the minimization of its makespan. Clusters are also able to cooperate and tolerate degradation of their criteria, in order to optimize the global makespan.

Still in the field of grid computing, we might mention Apache Hadoop's resource manager: YARN (Yet Another Resource Negotiator) which is used to manage system resources and schedule tasks Hosni et al. (2022). It is responsible for dynamically allocating resources to applications according to their needs. The global ResourceManager accepts jobs submitted by users, schedules them and allocates resources to them. There is also an ApplicationMaster, created for each application, responsible for negotiating resources for the application.

Another specific example is discussed in Belaid et al. (2012). The authors consider the packaging problem in the hygiene products industry. Batches of liquid products are temporarily stored in tanks before being packaged. Customer orders are made up of different product types and brands. At each changeover, the tanks and all packaging line components are cleaned using chemicals that are both harmful to the environment and costly. To minimize production costs, a global criterion is applied to all orders to optimize the use of resources and reduce toxic waste. Customers are agents to whom criteria are applied to meet delivery deadlines.

In this paper, we extend the studies presented in Sadi et al. (2014) by considering two-agent scheduling problems with interfering job sets on identical parallel machines, with makespan and total completion time criteria. A total of four different NP-hard problems are considered.

The remainder of this paper is organized as follows. First, we review in Sect. 2 the recent literature about multi-agent scheduling problems on parallel machines. The addressed problems are presented in Sect. 3 with preliminary results. Two generic Mixed Integer Linear Programming formulations are introduced in Sect. 4: the first is based on time-indexed variables and the second on the assignments of jobs to machines. In Sect. 5, a Genetic Algorithm is presented with two generic encoding schemes. In Sect. 6, the proposed methods are evaluated and compared. Finally, conclusion and future research directions are proposed in Sect. 7.

2 Survey of the literature

In this section, we review multi-agent scheduling problems on parallel machines. The papers are classified according to the machine environment, namely identical parallel machines, uniform parallel machines and unrelated parallel machines.

Identical parallel machines

In this environment, there are $m \geq 2$ identical parallel machines, i.e., the processing times of jobs do not depend on the machines. Each job must be scheduled on a single machine, with or without preemption.

In Balasubramanian et al. (2009), the authors are the first to consider a multi-agent scheduling problem in an identical parallel machines environment. In the problem they consider, the jobs belong to two disjoint sets, and each set has its own criterion to be minimized (the makespan or the total completion time). The authors proposed a time-indexed integer programming formulation, an iterative heuristic and an efficient genetic algorithm to generate the set of non-dominated solutions.

Then, several studies assume that the objective function of agent B has to be kept at or below a given value Q , whereas the objective of agent A has to be minimized. In Wan et al. (2010), the authors consider a two-agent scheduling problem on two identical parallel machines. Preemption is allowed, the jobs of agent A have controllable processing times, and the objective function of agent B is a regular function (i.e., a nondecreasing function with respect to its jobs completion times). The objective is to minimize the total completion time plus a compression cost of agent A while keeping the objective function of agent B below or at a fixed value. The authors propose polynomial-time algorithms for special cases when processing times and compression costs of agent A are agreeable. In Leung et al. (2010), the authors are interested in finding a schedule minimizing the objective of agent A while keeping the objective of agent B below or at a fixed level Q . When preemption is allowed and without release dates, the problems with objective $(f_{\max}^A, f_{\max}^B)$ can be solved in polynomial time and with objective functions $(\sum C_j^A, f_{\max}^B)$ the case where $m = 2$ can be solved in polynomial time. In Zhao and Lu (2013), the authors consider two two-agent scheduling problems on identical parallel machines. The objective is to minimize the makespan (or the total completion time) of agent A subject to an upper bound on the makespan of agent B. For both problems, they prove the NP-hardness and give a pseudo-polynomial-time algorithm and a FPTAS. The same authors present in Zhao

and Lu (2016) two approximation algorithms for the first problem, one for the two-machine version with running time $O(n \log(n))$ and another for $m > 2$ with the performance ratio of $2 - 1/m$ with a running time in $O(n)$. In Lee et al. (2016), the authors study a weighted version of the second problem addressed in Zhao and Lu (2013). The objective is to minimize the total weighted completion time of jobs of agent A, given a bound for the maximum completion time of jobs of agent B. They propose a branch-and-bound approach for small instances and a genetic algorithm to obtain approximate solutions on larger instances.

Regarding multi-agent scheduling (more than two agents), in Zhao et al. (2016) the authors study consider k agents on two identical parallel machines, with the objective to minimize the makespan of each agent. They propose an approximation algorithm whose performance ratio for the agent that has the i^{th} smallest completion time is no more than $i + 1/6$, and they show that this ratio is tight. For more than two machines, in Gu et al. (2018) the authors give an approximation algorithm based on LPT rule with a performance ratio vector equal to $(1 + (1/3 - 1/3m), 2 + (1/3 - 1/3m), \dots, K + (1/3 - 1/3m))$, when considering the agents in the increasing order of their associated makespan.

Some references dealt with rejection costs. In Choi and Park (2017), the authors address a two-agent scheduling problem on identical parallel machines, where the jobs of the second agent have time windows, and the number of jobs of the second agent processed simultaneously is restricted although some machines are available. Their objective is to minimize the total completion time for the first agent, while maintaining the total weight of the selected jobs of the second agent at or above a given threshold λ_i on each machine m_i . The authors study the complexity of several special cases with unit processing time for the first agent. They consider restrictions on the weight and the duration of agent 2, the number of machines and the maximum number of simultaneously processed jobs of agent 2. In Li and Lu (2017), the authors consider two identical parallel machines and the possibility to accept or to reject a job. A rejection of job j occurs a cost e_j . The objective function of agent k is equal to a general cost function f^k plus the total rejection cost $\sum e_j^k$. The authors studied several combinations of cost functions f^k ($C_{\max}^k, L_{\max}^k, \sum C^k, \sum w_j U_j^k$) and propose dynamic programming algorithms and fully polynomial-time approximation schemes.

More recently, in Yu et al. (2018) the authors consider a two-agent problem and a two identical parallel machines environment. A job can be processed by any of the two machines, or by only one predefined machine. The objective function is to minimize the total tardiness of the jobs of agent A and the makespan of the jobs of agent B. The authors compare several processing strategies, illustrated on

some examples. In Pei et al. (2020), the authors consider a competitive two-agent parallel-batching scheduling problem with aging effect on parallel machines. A batching machine can process several jobs at a time with a limited capacity. The processing time of jobs depends on their position (the higher the position, the greater the processing time, following to a linear increasing function). The objective is to minimize the makespan of agent A with the constraint that the makespan of agent B is no more than a given threshold. The authors propose a heuristic algorithm, a Bacterial Foraging with Variable Neighborhood Search (BF-VNS) algorithm, and compare this algorithm to other well-known algorithms.

Dealing with an interfering scenario, Sadi and Soukhal (2017) consider scheduling of equal-size independent jobs with identical processing times where a global agent has to minimize its objective function and the objective function of each agent is bounded. The authors mainly consider the total number of tardy jobs as the global and the individual objective functions. They propose complexity results.

Uniform parallel machines

In this type of environment, each machine may have its own operating speed which do not depend on the processed jobs, i.e., the processing time of a job only depends on the machine's speed.

In Elvikis et al. (2009), the authors consider a uniform parallel machines environment, jobs with identical processing times, and the goal is to enumerate the set of non-dominated solutions where a general objective function f^A is minimized for agent A and $C_{\max}^B$ for agent B. The authors propose some polynomial-time algorithms for several functions f^A such as $\sum U_j^A, \sum W_j C_j^A, L_{\max}^A$, etc. Later, in Elvikis and T'Kindt (2014) the authors consider the same model where both objective functions are of type $f_{\max}$.

In Gerstl and Mosheiov (2013), the authors consider the case where all the jobs of both agents have identical and unitary processing times and a common due date, in an identical and a uniform machines environment. The objective of agent A is to minimize the total weighted earliness–tardiness, and the objective of agent B is to keep the deviation from the common due date under a given threshold. The problems are similar to a linear assignment problem and are solved in polynomial time.

More recently, in Liu et al. (2018) the authors consider a two-agent scheduling problem on parallel machine with adjustable operating speed. The jobs belong to two disjoint sets, each one associated with an agent. The objective of the first agent is to minimize the total weighted tardiness, whereas the objective of the second agent is to minimize the total completion time. The authors want to determine a schedule that minimizes the objective of each agent, while maintaining the peak power below or at a fixed

level. Therefore, the operating speed of the machines is part of the decision since it affects the peak power. They propose a NSGA-II algorithm and an alternative genetic algorithm named Domination Number-based Genetic Algorithm (DNGA) to generate a set of non-dominated solutions.

In Gu et al. (2019), the authors consider a k -agent problem and a two-uniform parallel machines environment. The objective of each agent is to minimize its makespan. An approximation algorithm based on LPT rule is proposed. The authors show that the i^{th} completed agent is no more than $i + (\sqrt{17} - 3)/4$ times his/her optimal makespan. Some examples are proposed to illustrate the result.

Unrelated parallel machines

The performances of the machines depend on the processed jobs.

In Sadi et al. (2014), the authors consider K -agent scheduling problem in the environment of a single machine, identical or unrelated parallel machines, and preemption allowed or not. Each agent objective function cannot exceed a given threshold and the goal is to minimize a global objective function. The authors proposed a network flow approach for the case of preemption and unrelated parallel machines.

In Yin et al. (2016), the authors consider just-in-time scheduling problems involving two agents that compete for the use of m unrelated parallel machines for processing their jobs. The objective of agent A is to maximize the weighted number of its just-in-time jobs, which are the jobs completed exactly at their due date. The authors study two problems depending on the objective function of agent B and find the Pareto optimal solutions. The first problem, in which agent B wants to maximize its maximum gain, can be solved in polynomial time. The second problem, in which the objective of agent B is to maximize the weighted number of its just-in-time jobs, is proved ordinary NP-hard. The authors present a pseudo-polynomial-time algorithm and an FPTAS for finding an approximate Pareto optimal solution.

In Yin et al. (2019), the objective is to minimize the total completion time of the jobs of one agent while keeping the weighted number of tardy jobs of the second agent within a given limit. The authors design a branch-and-price algorithm embedding an in-out column generation scheme.

Summary

Table 1 summarizes the related works, and Table 2 lists the abbreviations which are used in Table 1.

First, we can notice that most of the existing studies consider competing scenarios, i.e., the case where the set of jobs of the agents are disjoint. Instead, in our work, we are interested in interfering scenarios, i.e., we have a global objective function. Then, we can see that several studies are interested

in minimizing the objective function of agent A while keeping the objective function of agent B below or at a fixed value. In our contribution, we want to minimize both objectives and we investigate methods to enumerate and approximate the Pareto front. Finally, concerning the objective functions, the closest study is in Balasubramanian et al. (2009). The authors considered two agents, and each set has its own criterion to be minimized (the makespan or the total completion time). However, in our case, the set of jobs of agents are not disjoint since agent A has the whole set of jobs. We can therefore derive four original and different NP-hard problems depending on the objective of each agent.

3 Problem definition and first results

3.1 Definitions

We consider two agents A and B . Agent A is a global agent; it means that it is associated with the whole set of jobs, denoted by $\mathcal{J}^A = \{J_1, J_2, \dots, J_{n_A}\}$, and agent B is associated with a subset of jobs denoted by $\mathcal{J}^B$. We denote by n_A the cardinality of $\mathcal{J}^A$ and n_B the cardinality of $\mathcal{J}^B$. We suppose that $\mathcal{J}^B = \{J_1, \dots, J_{n_B}\}$ and $\mathcal{J}^A \setminus \mathcal{J}^B = \{J_{n_B+1}, \dots, J_{n_A}\}$. The machine environment is composed of m identical parallel machines denoted by $M_1, \dots, M_m$ that are always available. Preemption is not allowed, and each machine can process only one job at a time. The processing time of J_j is denoted by p_j , and C_j is its completion time $\forall j, 1 \leq j \leq n_A$. We assume that all jobs are available at time zero.

We denote the makespan of the jobs of agents A and B by $C_{\max}^A = \max_{1 \leq j \leq n_A} C_j$ and $C_{\max}^B = \max_{1 \leq j \leq n_B} C_j$. Similarly, $\sum C_j^A = \sum_{j=1}^{n_A} C_j$ and $\sum C_j^B = \sum_{j=1}^{n_B} C_j$ are the total completion times of the jobs of agents A and B .

We use the three-field notation $\alpha|\beta|\gamma$ of scheduling problems introduced in T'kindt and Billaut (2006) for multicriteria scheduling problems and extended in Agnetis et al. (2014) to multi-agent scheduling problems. The problems we address in this paper are denoted by $Pm|IN|f^A, f^B$ with $f \in \{C_{\max}, \sum C_j\}$, i.e., the four two-agent scheduling problems that we consider are:

$$\begin{aligned} \mathcal{P}(C^B C^A): Pm|IN|C_{\max}^A, C_{\max}^B \\ \mathcal{P}(S^B C^A): Pm|IN|C_{\max}^A, \sum C_j^B \\ \mathcal{P}(S^B S^A): Pm|IN|\sum C_j^A, \sum C_j^B \\ \mathcal{P}(C^B S^A): Pm|IN|\sum C_j^A, C_{\max}^B \end{aligned}$$

These problems are NP-hard (Sadi et al. 2014). As for many multicriteria scheduling problems, our goal is to generate the set of non-dominated solutions for each problem instance. We recall that a feasible solution σ is a *non-dominated solution*, also called a weak Pareto solution, if

Table 1 Multi-agent scheduling problems on parallel machines

Reference	Problem	Methods	Scenario
Balasubramanian et al. (2009)	$P C_{\max}^A, \sum C_j^B$	IP, H, GA	CO
Wan et al. (2010)	$P2 pmtn, ctrl \sum(C_j^A + co_j^A x_j^A), f_{\max}^B \leq Q$	Poly-A	CO
Leung et al. (2010)	$Pm pmtn f_{\max}^A, f_{\max}^B \leq Q$	Poly-A	CO
	$P2 pmtn \sum C_j^A, f_{\max}^B \leq Q$		
Zhao and Lu (2013)	$Pm C_{\max}^A, C_{\max}^B \leq Q$	Pseudo-A	CO
	$Pm \sum C_j^A, C_{\max}^B \leq Q$	FPTAS	CO
Zhao and Lu (2016)	$Pm C_{\max}^A, C_{\max}^B \leq Q$	Approx	CO
Lee et al. (2016)	$Pm \sum w_j C_j^A, C_{\max}^B \leq Q$	BB, GA	CO
Zhao et al. (2016)	$P2 C_{\max}^1, \dots, C_{\max}^k$	Approx	CO
Gu et al. (2018)	$Pm C_{\max}^1, \dots, C_{\max}^k$	Approx	CO
Choi and Park (2017)	$Pm o \sum C_j^A : W_i^B \geq \lambda_i$	Complexity analysis	CO
Li and Lu (2017)	$P2 rej f^A + \sum e_j^A, f^B + \sum e_j^B \leq Q$	DP, FPTAS	CO
Yu et al. (2018)	$P2 \sum T_j^A, C_{\max}^B$	H	CO
Pei et al. (2020)	$Pm P - batch, p_{j(r)}^x = a + r\beta_j^x C_{\max}^A, C_{\max}^B \leq Q$	H	CO
Sadi and Soukhal (2017)	$Pm p_j = p f^0, f^1 \leq e_1, \dots, f^K \leq e_K$	Complexity analysis	INTERF
	$Pm p_j = p \sum U_j^0, \sum U_j^1 \leq e_1, \dots, \sum U_j^K \leq e_K$		
Elvikis et al. (2009)	$Q p_j = p f^A, C_{\max}^B$ with $ f^A \in \sum U_j^A, \sum W_j C_j^A, L_{\max}^A, \dots$	Poly-A	CO
Elvikis and T'Kindt (2014)	$Q p_j = p f_{\max}^A, f_{\max}^B$	Poly-A	CO
Gerstl and Mosheiov (2013)	$Pm p_j = 1, d_j = d \sum(\alpha_j^A E_j^A - \beta_j^A T_j^A), \max(\alpha_j^B E_j^B - \beta_j^B T_j^B) \leq Q$	Poly-A	CO
	$Qm p_j = 1, d_j = d \sum(\alpha_j^A E_j^A - \beta_j^A T_j^A), \max(\alpha_j^B E_j^B - \beta_j^B T_j^B) \leq Q$		
Liu et al. (2018)	$Qm \sum w_j T_j^A, \sum C_j^B, Power_{\max} \leq Q$	GA	CO
Gu et al. (2019)	$Q2 C_{\max}^1, C_{\max}^2, \dots, C_{\max}^k$	Approx	CO
Sadi et al. (2014)	$Pm pmtn \sum f^0, \sum f^k \leq e^k$	Poly-A	INTERF
	$Rm pmtn f_{\max}^0, f_{\max}^k \leq e^k$		
Yin et al. (2016)	$Rm \sum w_j J T_j^A, \max w_j J T_j^B$	Poly-A	CO
	$Rm \sum w_j J T_j^A, \sum w_j J T_j^B$	Pseudo-A, FPTAS	
Yin et al. (2019)	$Rm \sum C_j^A, \sum w_j U_j^B \leq Q$		CO

Table 2 List of abbreviations for table 1

IP	Integer programming
H	Heuristic
GA	Genetic algorithm
DP	Dynamic programming algorithm
Poly-A	Polynomial-time algorithm
Pseudo-A	Pseudo-polynomial-time algorithm
FPTAS	Fully polynomial-time approximation scheme
Approx	Approximation algorithm with performance ratio
BB	Branch-and-bound
BF-VNS	Bacterial foraging with variable neighborhood search
CO	Competing scenario (jobs belong to disjoint sets)
INTERF	Interfering scenario (presence of a global objective)

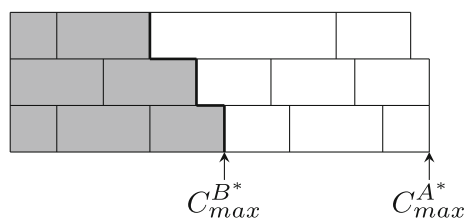


Fig. 1 Structure of non-dominated solution for the $Pm|IN|C_{\max}^A, f^B$

there is no other feasible solution better than or equal to σ for all of the criteria, and σ is strictly better than any other solution for at least one criterion.

It is worthwhile to note that an optimal solution for the $Pm|IN, f^B \leq Q_B|f^A$ problem (also denoted by $Pm|IN|\varepsilon(f^A/f^B)$) where Q_B is an upper bound on the agent B criteria leads to a weak Pareto solution. This approach is known in the literature as the ε -constraint approach.

3.2 Preliminary results

In this section, some dominance conditions related to the structure of non-dominated Pareto optimal solutions are proposed.

Proposition 1 *If problem $Pm|IN|\varepsilon(C_{\max}^A/f^B)$ ($f^B \in \{C_{\max}^B, \sum C_j^B\}$) has a feasible solution, then there is an optimal solution where on any machine, the jobs in $\mathcal{J}^B$ are executed before the jobs in $\mathcal{J}^A \setminus \mathcal{J}^B$.*

Proof Because the global objective function is to minimize the makespan and because the order of the jobs on each machine has no impact on this value, it is always possible to schedule the jobs of agent B before the jobs that are not in B, without modifying the objective function value for agent A. $\square$

Therefore, for problems $Pm|IN|C_{\max}^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$, any non-dominated vector in the criteria space matches a solution with a structure similar to the one depicted in Fig. 1, where the jobs in $\mathcal{J}^B$ are in black.

Proposition 2 *If problem $Pm|IN|\varepsilon(\sum C_j^A/C_{\max}^B)$ has a feasible solution, then there is an optimal solution where on each machine we have the following properties:*

1. all jobs scheduled before the last job of $\mathcal{J}^B$ (included) are sequenced in SPT order;
2. the jobs in $\mathcal{J}^A \setminus \mathcal{J}^B$ are sequenced in SPT order.

Proof Suppose there exists an optimal solution S^* where on one machine M_i one of these properties is not satisfied. If the first property is not satisfied, then before the last job of $\mathcal{J}^B$ (included) the jobs are not scheduled according to SPT

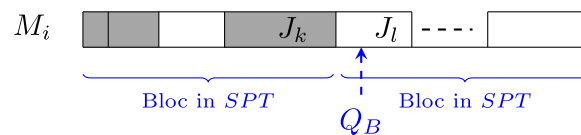


Fig. 2 Structure of an optimal solution on machine M_i for problem $\mathcal{P}(C^B S^A)$

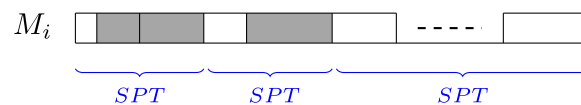


Fig. 3 Structure of an optimal solution on machine M_i for problem $\mathcal{P}(S^B S^A)$

order. Let consider two consecutive jobs J_k and J_l with J_k before J_l and $p_k > p_l$. Then swapping these two jobs does not violate the makespan constraint for the jobs of $\mathcal{J}^B$ and improves the total completion time. Notice that J_l can be the last job of $\mathcal{J}^B$ on M_i . If the second property is not satisfied, consider two jobs J_k and J_l in $\mathcal{J}^A \setminus \mathcal{J}^B$ with $p_k > p_l$ and J_k is scheduled before J_l . Then, swapping the two jobs would allow to decrease the agent A objective function, without violating the makespan constraint for the jobs of $\mathcal{J}^B$. In both cases, S^* cannot be optimal. $\square$

The overall pattern of the optimal solution on every machine is represented in Fig. 2 where the jobs in $\mathcal{J}^B$ are in black. The figure illustrates that it is possible to schedule a job J_k before J_l with $p_k > p_l$ because the jobs of agent B must be scheduled before the due date Q_B .

Proposition 3 *If problem $Pm|IN|\varepsilon(\sum C_j^A / \sum C_j^B)$ has a feasible solution, then there is an optimal solution where on each machine we have the following properties:*

1. the jobs in $\mathcal{J}^B$ are sequenced in SPT order;
2. the jobs in $\mathcal{J}^A \setminus \mathcal{J}^B$ are sequenced in SPT order.

Proof The proof is done by using a pairwise interchange argument. We first demonstrate that the jobs in $\mathcal{J}^B$ are sequenced in SPT order. Consider two jobs J_k and J_l in $\mathcal{J}^B$ with $p_k > p_l$, and J_k is scheduled before J_l . J_k and J_l are not necessarily consecutive and there may be some jobs of $\mathcal{J}^A \setminus \mathcal{J}^B$ and $\mathcal{J}^B$ scheduled in between. Swapping the two jobs will decrease the completion times of the jobs scheduled between J_k and J_l . Therefore, the two criteria are improved.

The same reasoning can be done with two jobs in $\mathcal{J}^A \setminus \mathcal{J}^B$. $\square$

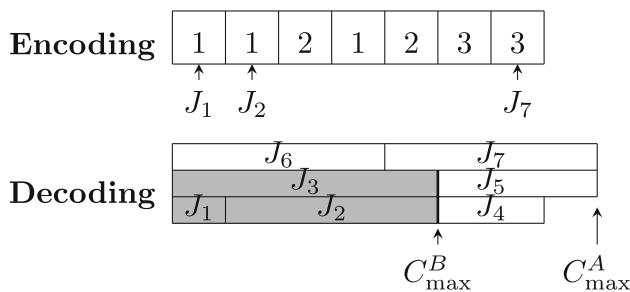
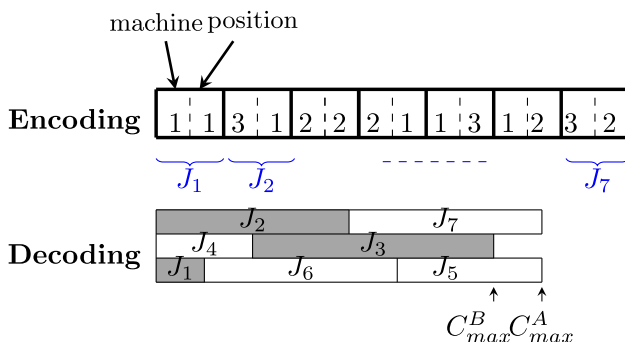
In this case, the overall pattern of the optimal solution is represented in Fig. 3, where the jobs in $\mathcal{J}^B$ are in black. Unlike the previous problems, here several SPT blocks can be constructed.

Table 3 Time-indexed models

Problem	Z	Constraints + (11)	# bin. var	# constraints
$\mathcal{P}(C^B C^A)$	$C_{\max}^A$ (2)	(4)–(9)	nT	$3n + n_B + T + 2$
$\mathcal{P}(S^B C^A)$	$C_{\max}^A$ (2)	(4)–(7), (10)	nT	$3n + T + 2$
$\mathcal{P}(S^B S^A)$	$\sum_{j=1}^n C_j$ (3)	(4)–(6), (10)	nT	$2n + T + 2$
$\mathcal{P}(C^B S^A)$	$\sum_{j=1}^n C_j$ (3)	(4)–(6), (8), (9)	nT	$2n + n_B + T + 2$

Table 4 Machine assignment models

Problem	Z	Constraints + (18)	# bin. var	# constraints
$\mathcal{P}(C^B C^A)$	$C_{\max}^A$ (12)	(14)–(17), (9)	nm	$mn + n + 2m + 1$
$\mathcal{P}(S^B C^A)$	$C_{\max}^A$ (12)	(14)–(16), (10)	nm	$mn + m + n + 1$
$\mathcal{P}(S^B S^A)$	$\sum w_j C_j$ (13)	(14), (15)	nm	$mn + n + 1$

**Fig. 4** Machine assignment Encoding/Decoding scheme for problems $Pm|IN|C_{\max}^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$. Jobs in $\mathcal{J}^B$ are in black**Fig. 5** Machine position encoding/decoding scheme for problems $Pm|IN|\sum C_j^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$. Jobs in $\mathcal{J}^B$ are in black

3.3 Number of Pareto solutions

It is known that the problems minimizing the total completion times as a global objective function (problems $\mathcal{P}(S^B S^A)$ and $\mathcal{P}(C^B S^A)$) admit an exponential number of Pareto solutions (Agnetis et al. 2014). In this section, we propose a discussion about the number of strict Pareto solutions for the problems minimizing the global agent makespan (problems $\mathcal{P}(S^B C^A)$ and $\mathcal{P}(C^B C^A)$). We show that these problems admit a pseudo-polynomial number of Pareto solutions. We first have the following theorem:

Theorem 1 The problem $Pm|IN|C_{\max}^A, C_{\max}^B$ has at most $p_{\max}$ Pareto solutions in the criteria space, with $p_{\max} = \max_{1 \leq j \leq n_A} p_j$, and this bound is reached.

Proof Let us denote by $C_{\max}^{A*}$ and by $C_{\max}^{B*}$ the optimal values of the makespan for agents A and B. We denote by σ_1 and σ_2 two extreme Pareto solutions that minimize, respectively, agent A and B objective functions; these solutions are obtained by solving the problems $Pm|IN, C_{\max}^A \leq C_{\max}^{A*} | C_{\max}^B \leq C_{\max}^{B*}$ and $Pm|IN, C_{\max}^B \leq C_{\max}^{B*} | C_{\max}^A \leq C_{\max}^{A*}$, respectively. We denote:

- the objective function values of σ_1 by $(C_{\max}^{A*}, C_{\max}^{B*}(\sigma_1))$;
- the objective function values of σ_2 by $(C_{\max}^A(\sigma_2), C_{\max}^{B*})$.

By definition, for any new Pareto solution σ , we have $C_{\max}^A(\sigma) \in [C_{\max}^{A*}, C_{\max}^A(\sigma_2)]$. We show that $(C_{\max}^A(\sigma_2) - C_{\max}^{A*}) < p_{\max}$. Let us denote by J_l the last job scheduled in σ_2 finishing at time $C_l = C_{\max}^A(\sigma_2)$. It is clear that all machines are busy in the interval $[0, C_{\max}^A(\sigma_2) - p_l]$ (otherwise, assigning J_l to another machine would decrease $C_{\max}^A(\sigma_2)$, which is not possible). Then, we have the following relation:

$$m(C_{\max}^A(\sigma_2) - p_l) \leq \sum_{\substack{j=1 \\ j \neq l}}^{n_A} p_j \quad (1)$$

Suppose that $(C_{\max}^A(\sigma_2) - C_{\max}^{A*}) \geq p_{\max}$.

Then, we have: $C_{\max}^A(\sigma_2) \geq C_{\max}^{A*} + p_{\max} \geq \frac{1}{m} \sum_{j=1}^{n_A} p_j + p_{\max}$ because $C_{\max}^{A*} \geq \frac{1}{m} \sum_{j=1}^{n_A} p_j$
 $\Rightarrow m C_{\max}^A(\sigma_2) \geq \sum_{j=1}^{n_A} p_j + m p_{\max}$
 $\Rightarrow m(C_{\max}^A(\sigma_2) - p_{\max}) \geq \sum_{j=1}^{n_A} p_j$.

We have $p_{\max} \geq p_l$, and so $m(C_{\max}^A(\sigma_2) - p_l) \geq m(C_{\max}^A(\sigma_2) - p_{\max})$.

Therefore, we have $m(C_{\max}^A(\sigma_2) - p_l) \geq \sum_{j=1}^{n_A} p_j > \sum_{\substack{j=1 \\ j \neq l}}^{n_A} p_j$, which is a contradiction with (1).

Table 5 Definition of the sequences used to generate the initial population for problems $Pm|IN|C_{\max}^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$

$$\begin{aligned} S_1^{\alpha, \beta} &= SPT^B(\alpha) // LPT^B(n_B - \alpha) // SPT^A(\beta) // LPT^A(n - n_B - \beta) \\ S_2^{\alpha, \beta} &= LPT^B(\alpha) // SPT^B(n_B - \alpha) // LPT^A(\beta) // SPT^A(n - n_B - \beta) \\ S_3^{\alpha, \beta} &= LPT^B(n_B) // LPT^A(\beta) // SPT^A(n - n_B - \beta) \\ S_4^{\alpha, \beta} &= LPT^B(n_B) // SPT^A(\beta) // LPT^A(n - n_B - \beta) \end{aligned}$$

Table 6 Definition of the sequences used to generate the initial population for problems $Pm|IN|\sum C_j^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$

$$\begin{aligned} S_1^{\alpha, \beta} &= SPT^B(\alpha) // LPT^B(n_B - \alpha) // SPT^A(\beta) // LPT^A(n - n_B - \beta) \\ S_2^{\alpha, \beta} &= LPT^B(\alpha) // SPT^B(n_B - \alpha) // LPT^A(\beta) // SPT^A(n - n_B - \beta) \\ S_3^{\alpha, \beta} &= LPT^B(n_B) // LPT^A(\beta) // SPT^A(n - n_B - \beta) \\ S_4^{\alpha, \beta} &= LPT^B(n_B) // SPT^A(\beta) // LPT^A(n - n_B - \beta) \end{aligned}$$

We deduce that for each Pareto solution σ , $C_{\max}^A(\sigma) \in [C_{\max}^{A*}, C_{\max}^{A*} + p_{\max})$. Then, at most $p_{\max}$ Pareto solutions are possible. $\square$

This bound is reached. Consider three jobs $\{J_1, J_2, J_3\}$ with $p_1 = 1, p_2 = 1$ and $p_3 = 2$ and two identical parallel machines, where the first two jobs are the agent B jobs. This instance admits two strict Pareto solutions in the criteria space which are $(C_{\max}^A, C_{\max}^B) \in \{(3, 1), (2, 2)\}$.

4 Integer programming formulation

We present in this section two Mixed Integer Linear Programming (MILP) formulations for solving the ϵ -constraint versions of the problems, when the objective function of agent B is bounded by a value denoted Q_B .

To generate the set of strict Pareto solutions, we solve the problems $Pm|f^B \leq Q_B|f^A$ with different values of Q_B . In a first step, Q_B is equal to the upper bound of the objective function f^B . The solution obtained is denoted by $(f^A, \widehat{f^B})$. In a second step, we solve the inverse problem $Pm|f^A \leq \widehat{f^A}|f^B$. The solution obtained is a strict Pareto, denoted by $(\widehat{f^A}, \widehat{f^{B'}})$, and added to the set of solutions. We then set $Q_B = \widehat{f^{B'}} - 1$ and iterate. When no feasible solution is obtained, the procedure stops. The MILP models are used iteratively, to obtain the exact Pareto front for each problem.

Two MILP models are presented, one with time indexed variables and one with positional variables. For each model, C_j is a continuous variable for the completion time of job J_j , and $C_{\max}^k$ is a continuous variable associated with the makespan of the jobs of agent k ($k \in \{A, B\}$). T denotes an upper bound to the maximum completion times.

4.1 Time-indexed formulation

We define a binary variable $x_{j,t}$ equal to 1 if job J_j starts its processing at time t , and 0 otherwise. The general formulation of the time-indexed MILP, called MILP-Time, is the

following.

$$\text{MIN } Z \quad s.t.$$

$$Z = C_{\max}^A \quad (2)$$

$$Z = \sum_{j=1}^n C_j \quad (3)$$

$$\sum_{t=0}^T x_{j,t} = 1 \quad \forall J_j \in \mathcal{J}^A \quad (4)$$

$$\sum_{j=1}^{n_A} \sum_{s=\max\{0, t-p_j+1\}}^t x_{j,s} \leq m \quad \forall t = 0, \dots, T \quad (5)$$

$$C_j - \sum_{t=0}^{T-p_j} (t + p_j) x_{j,t} = 0 \quad \forall J_j \in \mathcal{J}^A \quad (6)$$

$$C_{\max}^A \geq C_j \quad \forall J_j \in \mathcal{J}^A \quad (7)$$

$$C_{\max}^B \geq C_j \quad \forall J_j \in \mathcal{J}^B \quad (8)$$

$$C_{\max}^B \leq Q_B \quad (9)$$

$$\sum_{j=1}^{n_B} C_j \leq Q_B \quad (10)$$

$$x_{j,t} \in \{0, 1\} \quad \forall J_j \in \mathcal{J}^A, \quad t = 0, \dots, T \quad (11)$$

Constraints 4 impose that each job J_j starts at a given time t . Constraints 5 ensure that no more than m jobs are scheduled simultaneously. This avoids the jobs overlapping at any time t . Constraints 6 give the C_j values. Constraints 7 and 8 give the two agents' makespan values. Constraints 9 and 10 express different ϵ -approach bounds.

Each problem that we consider can be modeled by using a subset of these constraints.

In Table 3, we summarize different models. For each problem, we specify the criterion to be optimized and all the constraints taken into account.

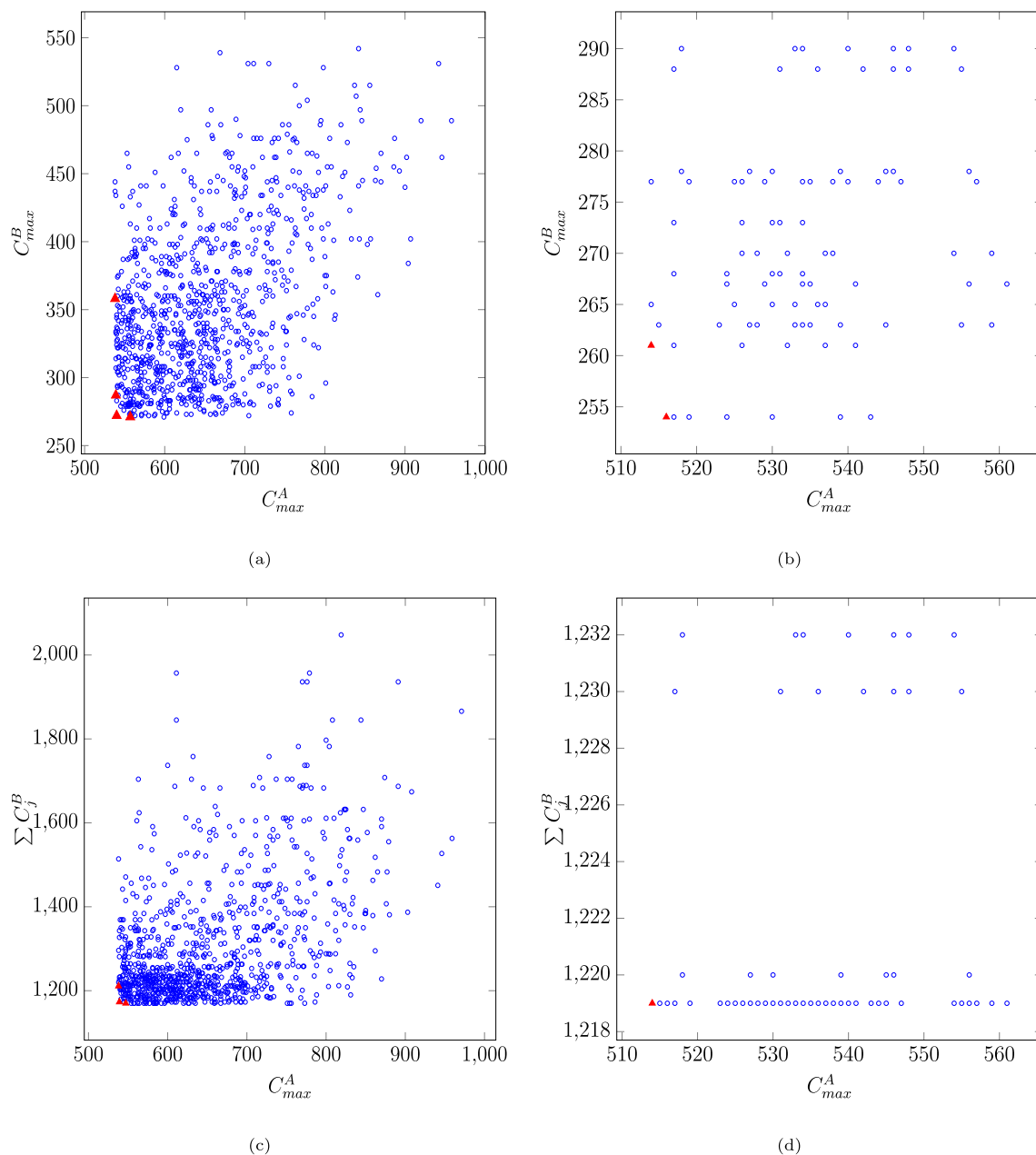


Fig. 6 Initial population for problems $Pm|IN|C_{\max}^A, f^B$ for one instance with $n_A = 20$, $n_B = 10$ and two machines. In Fig. 6a–c, the initial population are obtained from a random generation. In Fig. 6b, d, the initial population are obtained with Eq. (22). Red triangles are dominating solutions (Color figure online)

Note that this model determines the completion times but not the schedules. By considering each job as a time interval, finding the assignment of each job to the machines can be obtained by solving a coloration problem on an interval graph, which can be done in polynomial time (Olariu 1991; Kovalyov et al. 2007).

4.2 Machine assignment formulation

In this section, we present a mathematical formulation based on the assignments of jobs on the machines; this is inspired by Prop. 1 to solve the problems with $C_{\max}^A$ as an objective function (jobs in $\mathcal{J}^B$ are executed before jobs in $\mathcal{J}^A \setminus \mathcal{J}^B$). In this model, we define the binary variables $s_{i,j}$ equal to 1 if job J_j is executed on machine M_i , 0 otherwise. Moreover, we suppose that the jobs of agent B are indexed in SPT order,

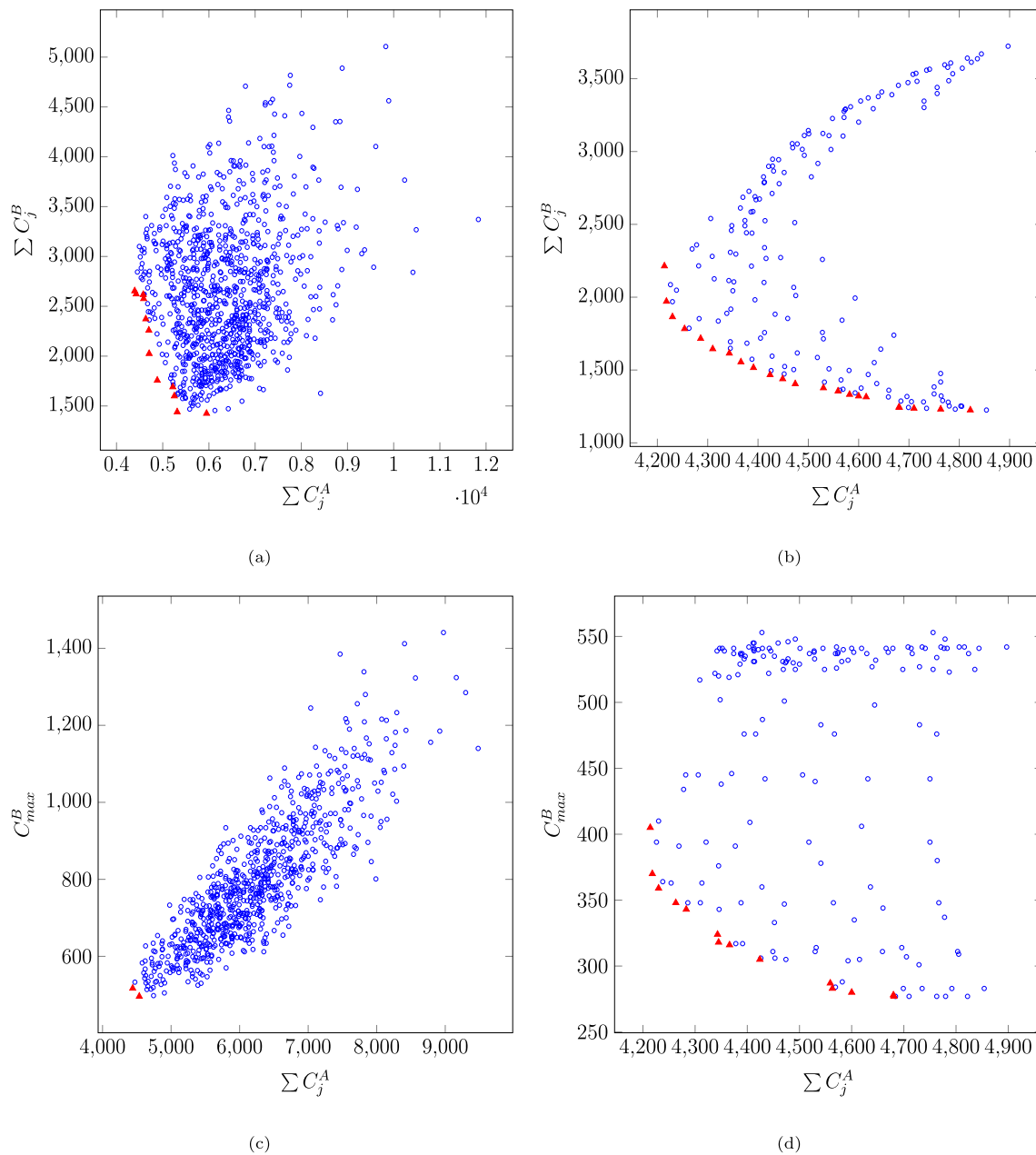


Fig. 7 Initial population for problems $Pm|IN|\Sigma C_j^A, f^B$ for one instance with $n_A = 20$, $n_B = 10$ and two machines. In Fig. 7a, c, the initial population are obtained from a random generation. In Fig. 7b,

d, the initial population are obtained with Eq. (23). Red triangles are dominating solutions (Color figure online)

Fig. 8 Example of one-point crossover with the machine position encoding (Color figure online)

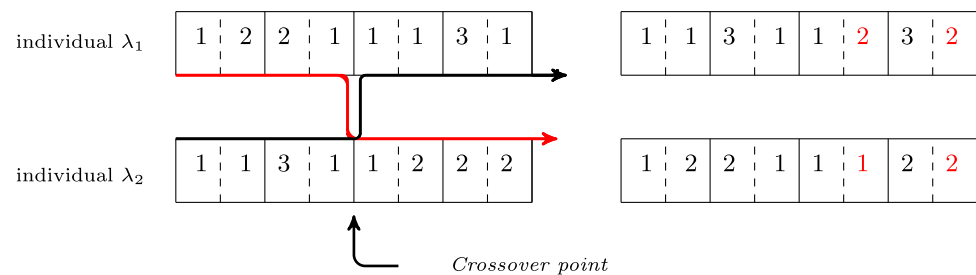
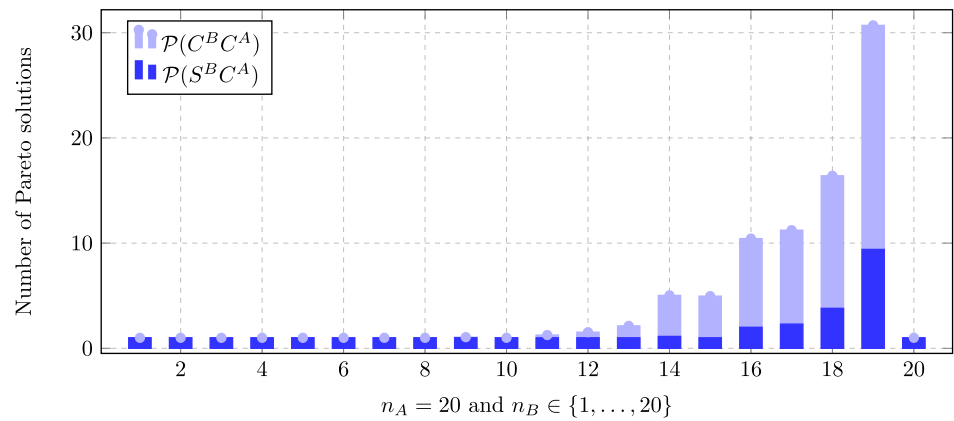
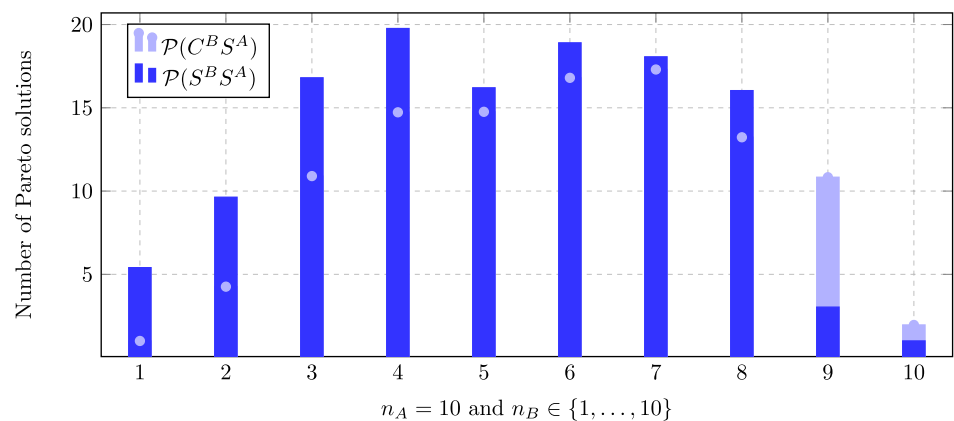


Fig. 9 Number of Pareto solutions depending on the number of agent B jobs (n_B)



(a) Problem $Pm|IN|C_{max}^A, f^B$



(b) Problem $Pm|IN|\sum C_j^A, f^B$

followed by the rest of the jobs indexed in SPT order as well. We have $p_1 \leq p_2 \leq \dots \leq p_{n_B}$, and $p_{n_B+1} \leq \dots \leq p_n$.

The general formulation of the machine assignment MILP, called MILP-Assign, is the following:

MIN Z s.t.

$$Z = C_{\max}^A \quad (12)$$

$$Z = \sum_{j=1}^n w_j C_j \quad (13)$$

$$\sum_{i=1}^m s_{i,j} = 1 \quad \forall J_j \in \mathcal{J}^A \quad (14)$$

$$C_j - \sum_{l=1}^j s_{i,l} p_l - (s_{i,j} - 1)T \geq 0 \quad \forall J_j \in \mathcal{J}^A, \quad \forall i = 1, \dots, m \quad (15)$$

$$C_{\max}^A - \sum_{j \in \mathcal{J}} s_{i,j} p_j \geq 0 \quad \forall i = 1, \dots, m \quad (16)$$

$$C_{\max}^B - \sum_{j \in \mathcal{J}^B} s_{i,j} p_j \geq 0 \quad \forall i = 1, \dots, m \quad (17)$$

$$s_{j,t} \in \{0, 1\} \quad \forall J_j \in \mathcal{J}^A, \quad t = 0, \dots, T \quad (18)$$

Constraints 14 ensure that each job J_j is assigned to exactly one machine. Constraints 15 rely on proposition 1 to calculate the job completion times: when J_j is not assigned to M_i ($s_{i,j}=0$), then the constraints hold anyway; otherwise, C_j is scheduled after the jobs in $\{J_1, \dots, J_{j-1}\}$ that are on the same machine. Constraints 16 and 17 give the two agents' makespan values.

This model can also be used to solve the problem $Pm|IN|\sum C_j^A, \sum C_j^B$ with a linear combination approach. In this case, jobs will be numbered following the Smith's rule (WSPT order): $w_1/p_1 \leq w_2/p_2 \leq \dots \leq w_n/p_n$. Indeed, let consider the problem $Pm|IN|\alpha \sum C_j^A + (1-\alpha) \sum C_j^B$ resulting from applying a linear combination on the agents' objective functions, with $\alpha \in [0, 1]$. This problem is equiva-

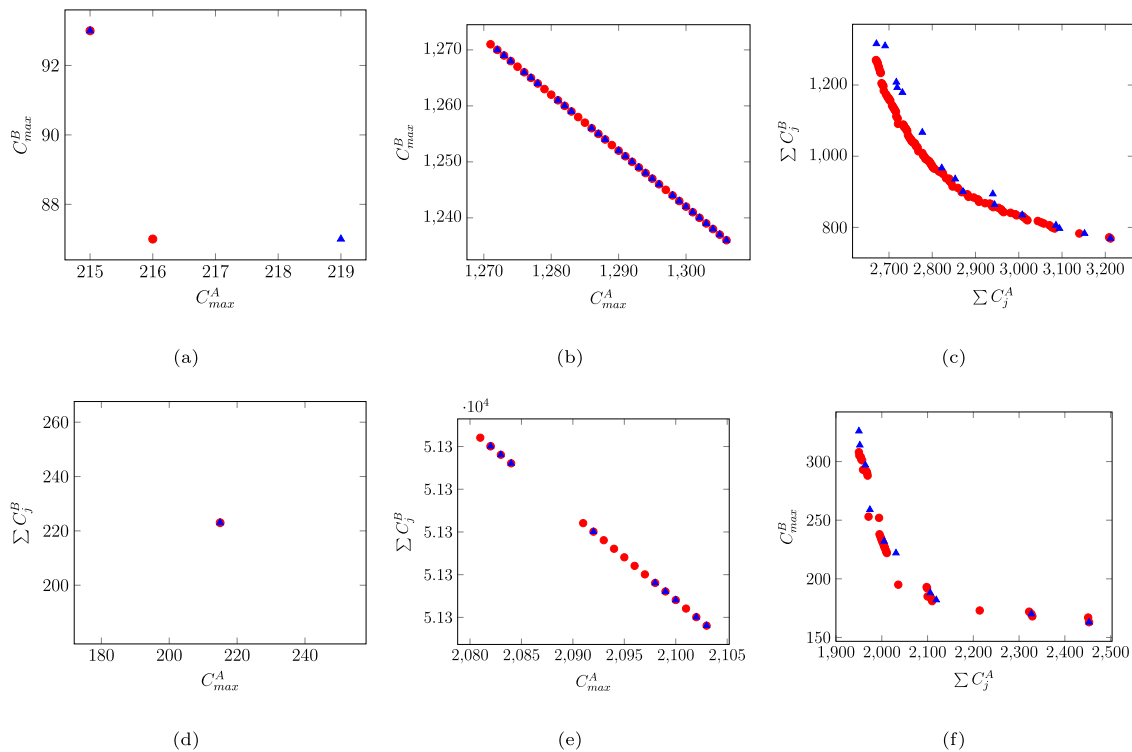


Fig. 10 Examples of Pareto fronts: red points are the exact Pareto solutions and blue triangles are approximate solutions (Color figure online)

lent to the classical scheduling problem $Pm||\sum w_j C_j$ Bruno et al. (1974) in which the weights w_j are given by:

$$w_j = \begin{cases} \alpha & \text{if } J_j \in \mathcal{J}^A \setminus \mathcal{J}^B \\ 1 & \text{if } J_j \in \mathcal{J}^B \end{cases}$$

The problem is then the same as the $Pm|IN|\sum_{J_j \in \mathcal{J}^B} C_j + \alpha \sum_{J_j \in \mathcal{J}^A \setminus \mathcal{J}^B} C_j$ problem.

We also know that given an assignment of the jobs on the machines, we can solve the problem $1||\sum w_j C_j$ on each machine by Smith's rule in less than $O(n \log(n))$ time. This explains why jobs are indexed according to the WSPT rule in this case.

As for the preceding model, we summarize in Table 4 different models for each problem, by specifying the criteria to optimize and the constraints to take into account.

Valid inequalities.

Some constraints can be introduced to break the symmetries Smith and Dh (2001); Jans (2009). Let us first impose that the maximum completion time of the jobs scheduled on machine M_i is greater than the maximum completion time of the jobs scheduled on machine M_{i+1} , for $i = 1$ to $m - 1$. This is

achieved by the following constraints:

$$\sum_{j=1}^n p_j s_{i,j} \geq \sum_{j=1}^n p_j s_{i+1,j}, \quad \forall i = 1, \dots, m - 1, \quad (19)$$

Trivial lower bounds have also been introduced in the models:

$$C_{\max}^A \geq \frac{1}{m} \sum_{j=1}^n p_j, \quad (20)$$

$$C_{\max}^B \geq \frac{1}{m} \sum_{j=1}^{n_B} p_j, \quad (21)$$

5 Genetic algorithm

In this section, we present a non-dominated sorting genetic algorithm called NSGA-II (Deb et al. 2002). This method is the commonly-used evolutionary algorithm for solving multiobjective optimization problems. Based on a genetic algorithm, NSGA-II uses a ranking selection method to emphasize current non-dominated solutions and a niching method to maintain diversity in the population. We refer to Konak et al. (2006) for a tutorial about multiple-objective optimization methods using genetic algorithms.

Table 7 Computational results for the problem $\mathcal{P}(C^B C^A)$ and $n_B = \lfloor n_A/2 \rfloor$

m	n_A	MILP-Time	MILP-Assign	$ \mathcal{S}^* $	NSGA-II		Comparison				
		CPU	CPU		CPU	$ \mathcal{S} $	%sol	%exact	%wSol	GD	Dist
2	10	20.39	3.41	2.26	0.0152	2.00	81.66	71.27	17.22	1.23	0.14
	15	130.47	2.53	1.20	0.0160	1.60	55.00	53.33	42.22	2.20	0.08
	20	482.93	1.84	1.00	0.0172	1.23	66.66	66.66	28.33	1.63	0
	30	–	1.74	1.00	0.0278	1.20	76.66	76.66	23.33	0.48	0
	40	–	2.13	1.00	0.0677	1.13	76.66	76.66	21.11	0.28	0
	50	–	1.92	1.00	0.1614	1.06	83.33	83.33	16.66	0.16	0
3	10	7.83	4.81	2.23	0.0159	1.93	82.94	75.11	13.05	1.58	0.20
	15	49.79	5.43	1.66	0.0169	1.90	27.77	25.00	63.33	5.81	0.41
	20	230.5	5.82	1.33	0.0190	1.76	28.88	26.33	53.33	5.12	0.38
	30	–	3.73	1.00	0.0325	1.53	20.00	20.00	60.00	3.23	0
	40	–	3.76	1.00	0.0820	1.26	40.00	40.00	53.33	1.55	0
	50	–	4.16	1.00	0.2029	1.16	63.33	63.33	34.44	1.16	0

The propositions presented in Sect. 3 are used to design the population as well as to explore all the space of feasible solutions.

5.1 Encoding mechanisms

According to Prop. 1, we know that given the assignments of the jobs to the machines, we can compute the objective function values of a non-dominated solution for the $Pm||C_{\max}^A, f^B$ problems with $f^B \in \{C_{\max}^B, \sum C_j^B\}$. The first proposed encoding is based on a *machine-assignment* scheme. However, for the $Pm|IN|\sum C_j^A, \sum C_j^B$ problem, it has been proved that for a given assignment of jobs to the machines, it is possible to have an exponential number of non-dominated solutions (Huynh Tuong et al. 2012). Therefore, we propose a second encoding scheme called *machine position*, that is best suited for this problem and for the $Pm|IN|\sum C_j^A, C_{\max}^B$ problem.

5.1.1 Machine assignment encoding

An individual is a n -vector where each element j stores the number of the machine assigned to job J_j . Let consider an example with $m = 3$ machines, $n = 7$ jobs and $n_B = 3$. Let the processing times be equal to $\{p_1 = 1, p_2 = 4, p_3 = 5, p_4 = 2, p_5 = 3, p_6 = 4, p_7 = 4\}$. We remember that the first three jobs belong to $\mathcal{J}^B$, i.e., J_1, J_2 and J_3 . This is due to the way jobs are numbered in the problem definition. Figure 4 shows the encoding and decoding of a solution if the problem is of type $Pm|IN|C_{\max}^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$.

5.1.2 Machine position encoding

For a problem with n jobs and m identical parallel machines, the length of an individual is $2n$. Each cell j is composed of

two elements: the machine assigned to J_j and its position on the machine. This encoding is used when $f^A = \sum C_j^A$ is the objective function of agent A . Figure 5 shows an example of encoding and decoding for the previous instance, with another assignment definition.

5.2 Generation of the initial population

There are several ways to generate an initial population $\mathcal{P}^0$ for an evolutionary algorithm: random generation, local search methods, metaheuristics, etc. We often choose a method that guarantees diversity in the initial population. Here, the initial population is composed of four sequences that are heuristically generated plus some randomly generated sequences.

The sequences are built using two heuristic algorithms. The first heuristic sorts the jobs in order of their non-decreasing processing times (SPT rule) and assigns each job to the first available machine (FAM rule); this algorithm is known in the literature to solve the $Pm||\sum C_j$ optimally. The second heuristic sorts the jobs in their non-increasing processing times order (LPT rule) and assigns them in FAM order, which is a well-known heuristic method for the $Pm||C_{\max}$ problem.

We use the notation $R^k(\alpha)$ $k \in \{A, B\}$ when the first α unscheduled jobs of agent k are scheduled according to rule R . The notation $a//b$ stands for the concatenation of a and b . Each problem has its own specificities that can be considered when generating the initial population (see Sect. 3). For this reason, the generation of the initial population is problem dependent. In the following, α belongs to $[1, n_B]$ and β belongs to $[(n_B + 1), n_A]$.

Table 8 Comparison between exact and approximate Pareto fronts for problem $\mathcal{P}(C^B C^A)$ where $n_B > \lfloor n_A/2 \rfloor$

m	n_A	n_B	MILP-Assign		NSGA-II		Comparison				
			CPU	$ S^* $	CPU	$ S $	%sol	%exact	%wSol	GD	Dist
2	10	7	13.02	6.36	0.016	4.83	92.14	76.14	5.97	0.68	0.04
	10	8	18.86	11.36	0.016	7.76	90.57	66.35	6.38	0.61	0.06
	10	9	37.12	22.93	0.016	12.90	98.07	62.02	1.92	0.02	0.02
	20	15	13.96	4.96	0.017	3.86	89.51	76.11	10.48	0.45	0.06
	20	17	22.53	11.23	0.017	7.76	98.65	76.71	1.349	0.37	0.03
	20	19	50.32	30.70	0.017	18.90	99.20	66.71	0.79	0.00	0.01
	30	25	17.15	6.70	0.020	5.36	95.21	89.50	7.00	0.35	0.01
	30	27	28.58	14.10	0.018	10.03	99.00	83.11	0.99	0.07	0.01
	30	29	44.32	27.90	0.017	18.06	98.64	71.00	1.35	0.01	0.01
	40	35	18.92	6.16	0.024	5.36	95.13	87.79	4.30	0.65	0.06
	40	37	31.86	11.73	0.020	9.36	99.33	87.45	0.66	0.00	0.01
	40	39	37.83	25.10	0.017	15.93	99.39	72.28	0.60	0.00	0.01
3	10	7	15.26	5.70	0.016	4.26	71.12	54.86	24.96	1.26	0.08
	10	8	23.66	9.40	0.017	6.53	61.27	45.51	38.77	1.18	0.07
	10	9	24.73	10.86	0.017	6.73	55.31	39.95	48.58	1.04	0.07
	20	15	56.07	10.90	0.018	5.20	31.85	17.19	82.46	1.51	0.13
	20	17	60.89	19.33	0.017	8.13	29.53	13.55	93.13	1.43	0.08
	20	19	48.86	20.76	0.017	9.63	43.19	23.21	72.60	0.88	0.07
	30	25	307.58	9.16	0.022	5.06	64.09	43.92	43.43	2.54	0.10
	30	27	63.37	15.46	0.019	6.73	49.16	24.35	61.28	0.90	0.12
	30	29	49.60	19.06	0.018	7.80	51.06	30.10	60.05	0.83	0.07
	40	35	53.15	9.46	0.028	5.76	58.43	43.03	46.68	1.33	0.09
	40	37	66.98	14.66	0.021	6.60	57.73	27.65	51.49	0.92	0.11
	40	39	46.43	17.06	0.018	7.70	51.53	30.15	60.58	0.80	0.08

- For problems $Pm|IN|C_{\max}^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$, the initial population is:

$$\mathcal{P}^0 = \bigcup_{\alpha, \beta} \bigcup_{l=1,4} S_l^{\alpha, \beta} \quad (22)$$

and the sequences $S_{l=1,4}^{\alpha, \beta}$ are given in table 5.

- For problems $Pm|IN|\sum C_j^A, f^B$ with $f^B \in \{C_{\max}^B, \sum C_j^B\}$, the initial population is:

$$\mathcal{P}^0 = \bigcup_{\alpha, \beta} \bigcup_{l=5,8} S_l^{\alpha, \beta} \quad (23)$$

and the sequences $S_{l=5,8}^{\alpha, \beta}$ are given in table 6.

Note that for problem $Pm|IN|\sum C_j^A, \sum C_j^B$, the SPT rule generates a non-dominated solution that can be obtained with appropriate values of α and β .

In Fig. 6 and Fig. 7, we present in the criteria space some examples of populations generated using the techniques discussed previously. The populations on the left (Figs. 6a, c, 7a, c) result from a random generation; the remainder on the

right are from the combination of SPT and LPT rules. In Fig. 6b, d the initial population are obtained with Eq. (22). In Fig. 7b, d the initial population are obtained with Eq. (23).

At first sight, the random method generates more individuals that are distributed sporadically in a larger interval. Indeed, the scales of the graphs on the left are larger than those on the right. For example, all the solutions in Fig. 7a belong to $[4200, 12000] \times [1100, 5000]$, and those in Fig. 7b belong to the interval $[4200, 4900] \times [1000, 3600]$. We can observe for problems with $C_{\max}$ as the global objective function, in Fig. 6, that at most four solutions dominate. For problems with $\sum C_j^A$ as the global objective function, more dominant solutions are generated using the heuristics (Fig. 7b, d).

5.3 Non-dominated sorting and ranking

The population of individuals is classified into successive non-dominated ranks or fronts, called rank 1, rank 2 and so on. This means that the solutions belonging to rank k are non-dominated among themselves and are dominated by the solutions of rank $k - 1$.

Table 9 Computational results for the problem $\mathcal{P}(S^B C^A)$ and $n_B = \lfloor n_A/2 \rfloor$

m	n_A	MILP-Time	MILP-Assign	NSGA-II			Comparison				
		CPU	CPU	$ S^* $	CPU	$ S $	%sol	%exact	%wSol	GD	Dist
2	10	6.83	3.92	1.33	0.015	1.26	88.33	83.33	13.33	0.68	0.09
	15	35.73	3.615	1.00	0.0163	1.06	90.00	90.00	10.00	0.65	0
	20	268.63	3.37	1.000	0.017	1.13	86.66	86.66	13.33	1.36	0
	30	2041.02	10.32	1.00	0.030	1.00	100	100	0	0	0
	40	–	56.05	1.00	0.071	1.00	100	100	0	0	0
	50	–	647.19	1.00	0.161	1.00	100	100	0	0	0
3	10	3.73	4.46	1.60	0.015	1.56	75.55	69.44	18.88	3.64	0.22
	15	18.42	4.45	1.10	0.017	1.36	35.00	33.33	61.66	5.82	0.08
	20	65.30	8.57	1.00	0.019	1.23	63.33	63.33	33.33	4.00	0
	30	785.8	118.33	1.00	0.036	1.00	96.66	96.66	3.33	0.03	0
	40	–	13,377	1.00	0.091	1.00	100	100	0	0	0

Table 10 Comparison between exact and approximate Pareto fronts for problem $\mathcal{P}(S^B C^A)$ where $n_B > \lfloor n_A/2 \rfloor$

m	n_A	n_B	MILP-Assign		NSGA-II		Comparison				
			CPU	$ S^* $	CPU	$ S $	%sol	%exact	%wSol	GD	Dist
2	10	7	4.08	1.70	0.014	1.46	96.66	84.44	2.50	1.19	0.12
	10	8	7.48	2.76	0.015	2.56	90.38	83.94	7.27	1.24	0.05
	10	9	10.20	4.26	0.015	3.83	91.02	82.74	8.30	0.20	0.06
	20	15	7.98	1.00	0.017	1.13	80.00	80.00	20.00	0.40	0.00
	20	17	20.61	2.30	0.016	1.86	88.03	84.76	13.63	0.41	0.00
	20	19	228.27	9.40	0.016	6.43	95.52	79.12	5.80	0.19	0.02
	30	25	1700.04	1.66	0.021	1.43	99.44	97.68	0.55	0.03	0.00
	30	27	7259.14	2.96	0.018	2.26	96.66	90.45	3.33	0.06	0.00
	30	29	28,355	8.40	0.016	5.50	95.11	81.09	5.76	0.21	0.017
3	10	7	9.16	2.76	0.015	2.23	81.16	68.54	9.55	1.46	0.25
	10	8	10.63	3.20	0.016	2.63	79.77	69.15	21.22	0.99	0.21
	10	9	18.15	5.06	0.015	3.93	90.85	72.77	6.97	0.62	0.12
	20	15	210.13	1.63	0.019	1.53	58.33	53.05	40.23	1.82	0.06
	20	17	1271.77	4.16	0.017	3.03	48.06	40.81	43.61	1.53	0.08
	20	19	27,713	13.03	0.016	6.33	94.62	65.19	5.56	0.20	0.03

Table 11 Computational results for problem $\mathcal{P}(S^B S^A)$ and $n_B = \lfloor n_A/2 \rfloor$

m	n_A	MILP-Time		MILP-Assign		NSGA-II		Comparison					
		CPU	$ S^* $	CPU	$nbSol$	CPU	$ S $	%Sol	%exact	%wSol	GD	Dist	$\mathcal{H}$
2	10	36.35	16.2	0.13	7.10	0.018	8.80	74.39	46.23	18.04	2.65	0.029	0.0017
	15	912.67	66.4	1.89	9.36	0.020	15.3	51.74	15.99	36.16	5.55	0.057	0.0049
	20	–	–	28.68	10.43	0.023	21.53	–	–	–	–	–	–
3	10	120.80	10.66	0.42	6.06	0.018	5.6	77.99	50.40	18.04	2.97	0.06	0.0019
	15	386.37	46.70	24.35	8.90	0.020	11.7	51.18	16.07	40.14	5.48	0.06	0.0038
	20	–	–	2151.72	10.26	0.024	18.2	–	–	–	–	–	–
4	10	55.56	6.33	0.53	5.36	0.018	4.53	81.74	62.13	17.41	1.85	0.04	0.0007
	15	183.53	33.2	100.49	7.66667	0.020	7.36	59.90	19.98	33.53	5.09	0.09	0.0040
	20	22672.7	116.56	–	–	0.024	14.56	38.69	5.51	59.29	6.97	0.087	0.0054

Table 12 Computational results for the problem $\mathcal{P}(C^B S^A)$ and $n_B = \lfloor n_A/2 \rfloor$

m	n_A	MILP-Time		NSGA-II		Comparison					
		CPU	$ S^* $	CPU	$ S $	%sol	%exact	%wSol	GD	Dist	$\mathcal{H}$
2	10	39.23	14.76	0.018	9.30	85.23	57.01	10.22	2.28	0.02	0.0015
	15	1331.77	41.16	0.020	15.80	51.60	22.76	46.56	6.96	0.03	0.0040
3	10	16.53	10.33	0.019	7.60	72.02	56.46	22.96	4.14	0.05	0.0027
	15	380.10	41.43	0.021	13.10	58.06	21.17	40.12	4.95	0.05	0.0063
4	10	9.02	6.23	0.019	5.06	79.55	69.11	21.00	5.02	0.05	0.0019
	15	125.13	24.63	0.021	10.40	49.71	23.63	44.945	4.54	0.10	0.0063
	20	1723.57	64.43	0.025	14.30	30.90	6.94	88.14	5.56	0.08	0.0105

This ranking procedure is called extensively in the NSGA-II algorithm. Several efficient versions of this procedure have been proposed in the literature D'Souza et al. (2010). To our knowledge, the best procedure is that of Kung et al. (1975); it is based on binary search and runs in $O(N_{\mathcal{P}} \log N_{\mathcal{P}})^K$ with $K \geq 4$ the number of criteria and $O(N_{\mathcal{P}} \log N_{\mathcal{P}})$ if $K \leq 3$, with $N_{\mathcal{P}}$ the size of population $\mathcal{P}$.

The ranks are also used in the definition of a probability associated with an individual. The best individuals have a rank equal to 1, and we would like to attribute to them the highest possible probability. The idea is to promote the best individuals for the improvement in the next populations. Let R_{λ} be the rank of individual λ and $R_{\max}$ the maximum rank. The expression of probability Pr_{λ} , of $\lambda \in \mathcal{P}$ is the following:

$$Pr_{\lambda} = \frac{R_{\max} - R_{\lambda}}{N_{\mathcal{P}} R_{\max} - \sum_{\ell \in \mathcal{P}} R_{\ell}}$$

We note that $\sum_{\lambda \in \mathcal{P}} Pr_{\lambda} = 1$.

5.4 Selection, crossover and mutation

Algorithm 1 presents the main steps of the genetic algorithm. First, the initial population is generated as explained in subsection 5.2 at line 1. Then, at each iteration the algorithm generates a new population until the maximum number of iterations is reached (line 3). Let $\mathcal{P}^k$ be the population being generated at iteration k , $\mathcal{C}^k$ be the set of individuals generated by the crossover operator, $\mathcal{M}^k$ be the set of individuals generated by the mutation operator (line 4). To obtain a child solution, one parent λ_1 is selected from the current population $\mathcal{P}^{k-1}$ (line 7). The following procedure is executed until the number of individuals generated from individual λ_1 is equal to $N_{\mathcal{P}^k} \times Pr_{\lambda_1}$ (line 19). A value ρ is randomly generated between 0 and 1 (line 9). If $\rho \leq \rho_{mut}$, a second parent λ_2 is selected randomly in $\mathcal{P}^{k-1}$, λ_1 and λ_2 are crossed, and

the two offsprings are added to population $\mathcal{C}^k$ (line 10 to line 13). Otherwise, individual λ_1 mutates and individual λ'_1 is added to population $\mathcal{M}^k$ (line 15 to line 17). To create λ'_1 , two jobs are randomly selected in the individual λ_1 and their characteristics are swapped (position and assignment).

Algorithm 1 Genetic algorithm

Require: r the number of crossover points, ρ_{mut} the probability of mutation, max_iter the maximum number of iteration, $N_{\mathcal{P}^k}$ the desired size of the population at iteration k

- 1: $\mathcal{P}^0 \leftarrow$ initial population obtained with Eqs. (22) or (23).
- 2: $k \leftarrow 1$
- 3: **while** $k \leq max_iter$ **do**
- 4: $\mathcal{P}^k \leftarrow \emptyset$, $\mathcal{C}^k \leftarrow \emptyset$, $\mathcal{M}^k \leftarrow \emptyset$, $i \leftarrow 1$
- 5: **repeat**
- 6: $nbChils \leftarrow 0$
- 7: $\lambda_1 \leftarrow$ the i^{th} individual in $\mathcal{P}^{k-1}$, assuming solutions in $\mathcal{P}^{k-1}$ are ranked.
- 8: **repeat**
- 9: $\rho \leftarrow$ a random number between 0 and 1
- 10: **if** $\rho \leq \rho_{mut}$ **then**
- 11: Select randomly λ_2 from $\mathcal{P}^{k-1}$
- 12: $\mathcal{C}^k \leftarrow \mathcal{C}^k \cup crossover(\lambda_1, \lambda_2, r)$
- 13: $nbChils \leftarrow nbChils + 2$
- 14: **else**
- 15: $\lambda'_1 \leftarrow mutation(\lambda_1)$
- 16: $\mathcal{M}^k \leftarrow \mathcal{M}^k \cup \lambda'_1$
- 17: $nbChils \leftarrow nbChils + 1$
- 18: **end if**
- 19: **until** $nbChils \geq N_{\mathcal{P}^k} \times Pr_{\lambda_1}$
- 20: $i \leftarrow i + 1$
- 21: **until** $|\mathcal{M}^k| + |\mathcal{C}^k| \geq N_{\mathcal{P}^k}$
- 22: $\mathcal{P}^k \leftarrow \mathcal{M}^k \cup \mathcal{C}^k$
- 23: $k \leftarrow k + 1$
- 24: **end while**

We explain now the X -point crossover between two individuals with the *machine position* encoding. Figure 8 illustrates the one-point crossover.

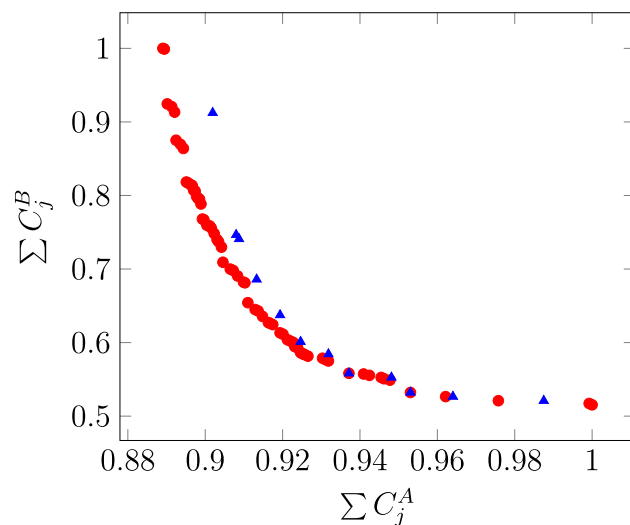
Let r be the number of crossover points ($r \leq n - 1$). Let us denote by c_1 the first individual resulting from a

Table 13 Mean criteria values and generational distance for problem $\mathcal{P}(C^B C^A)$ and $n_B = \lfloor n_A/2 \rfloor$

m	n_A	MILP-Assign		NSGA-II		GD
		$ C_{\max}^A $	$ C_{\max}^B $	$ C_{\max}^A $	$ C_{\max}^B $	
2	10	249.12	133.30	249.53	131.96	1.23
2	15	376.26	172.75	376.63	174.61	2.20
2	20	495.76	247.23	495.91	248.75	1.63
2	30	754.03	381.93	754.13	382.31	0.48
2	40	1067.23	539.86	1067.30	540.10	0.28
2	50	1249.60	627.76	1249.63	627.90	0.16
3	10	168.07	103.14	169.17	99.84	1.58
3	15	251.51	121.61	252.68	125.79	5.81
3	20	330.96	166.98	331.57	171.58	5.12
3	30	502.90	254.70	503.27	257.70	3.23
3	40	711.70	360.03	711.87	361.46	1.55
3	50	833.16	418.66	833.41	419.60	1.16

Table 14 Mean criteria values and generational distance for problem $\mathcal{P}(S^B S^A)$ and $n_B = \lfloor n_A/2 \rfloor$

m	n_A	MILP-Time		NSGA-II		GD
		$ \sum C_j^A $	$ \sum C_j^B $	$ \sum C_j^A $	$ \sum C_j^B $	
2	10	1165.64	423.88	1183.57	409.03	2.65
	15	2452.88	753.35	2530.60	689.62	5.55
3	10	867.79	331.35	875.04	325.70	2.97
	15	1767.77	569.15	1807.18	537.92	5.48
4	10	727.77	286.18	732.73	280.65	1.85
	15	1428.65	476.85	1463.67	451.76	5.09
	20	2309.68	835.22	2398.73	767.43	6.97

**Fig. 11** Normalized exact solutions (red points) and approximate solutions (blue triangles) for problem $\mathcal{P}(C^B C^A)$ of an instance with $m = 2$, $n_A = 15$ (Color figure online)

crossover between λ_1 and λ_2 . Child c_1 is composed of $r + 1$ parts denoted by $\beta_1 // \beta_2 // \dots // \beta_{r+1}$ with β_l the l^{th} part ($l = 1, \dots, r + 1$). Child c_1 inherits $\beta_1, \beta_3, \beta_5$, etc. from λ_1 , and the remaining parts from λ_2 . Child c_2 inherits β_2, β_4 , etc. from λ_1 , and its other parts from λ_2 . For c_1 , the assignments and positions of jobs are those of λ_1 ; the positions of jobs from λ_2 are repaired accordingly (the same order as in λ_2 and empty positions are filled). The process is similar for c_2 (see the positions in red in Fig. 8).

The crossover process is the same for the *machine-assignment* encoding.

6 Computational experiments

6.1 Data generation and configurations

The performance evaluation of the algorithms under study has been carried out with 360 instances, with a number of jobs $n_A \in \{10, 15, 20, 30, 40, 50\}$ and a number of machines $m \in \{2, 3\}$ (30 instances are generated per n_A). The jobs processing times have been generated using a discrete uniform distribution between 1 and 100.

The algorithms have been coded in C++ language and executed on a 2 Ghz AMD Epyc 7702 processor. Cplex version 12.9 is used to solve the MILP models with one thread and default parameters. The computation time limit has been fixed to 24 h for solving the 30 instances of a given n_A .

6.2 Preliminary study of the number of Pareto solutions

In this section, we generate 30 instances per value of n_A using the previous process, with $n_A \in \{10, 20\}$. These instances are used to compute the number of Pareto solutions obtained by the exact method, depending on the number of jobs of agent B . The results are represented in Fig. 9.

- Figure 9a shows that for problems with $C_{\max}^A$ as the global objective function, when $n_B \in \{1, \dots, 11\}$, only one Pareto solution is found, corresponding to an ideal solution. However, when the n_B value increases, the number of Pareto solutions also increases, and the maximum is always reached when $n_B = n_A - 1$. Consequently, in the following experiments, we first chose $n_B = \lfloor n_A/2 \rfloor$ then $n_B \in [90\%n_A, 99\%n_A]$, for problems with $C_{\max}^A$.
- Figure 9b shows that for problems minimizing $\sum C_j^A$ as the global objective function, the average number of Pareto solutions is important, and the maximum is reached when the n_B value is approximately $\lfloor n_A/2 \rfloor$. Hence, we take $n_B = \lfloor n_A/2 \rfloor$ for problems minimizing $\sum C_j^A$.

6.3 Measuring the performance of NSGA-II

For each instance, we have compared the exact front $\mathcal{S}^*$, generated by the MILP and the Pareto front $\mathcal{S}$ generated by NSGA-II algorithm. Figure 10 shows one approximated and one exact front for each problem.

Figure 10a, b shows the results of problem $\mathcal{P}(C^B C^A)$, Fig. 10d, e shows the results of problem $\mathcal{P}(S^A C^A)$, Fig. 10c shows the results of problem $\mathcal{P}(S^B S^A)$, and Fig. 10f shows the results of problem $\mathcal{P}(C^B S^B)$. According to the results presented in Sect. 6.2, we have fixed n_B to $n_A - 1$ in Fig. 10b, e, and to $n_A/2$ in the remaining figures. Clearly, more strict Pareto solutions are obtained when $n_B = n_A - 1$; in this case the NSGA-II generates a subset of the strict Pareto solutions. This figure shows also a crowded structure for problems $\mathcal{P}(S^B S^A)$ and $\mathcal{P}(C^B S^A)$.

6.3.1 Performance measures

As outlined in the previous section, the structure of the Pareto fronts is different from one problem to another; therefore, different measures have been defined. We denote by $\mathcal{S}^* = \{a_1, \dots, a_{|\mathcal{S}^*|}\}$ the exact Pareto front and by $\mathcal{S} = \{b_1, \dots, b_{|\mathcal{S}|}\}$ an approximated Pareto front. We categorize the performance measures into three classes.

Cardinality We calculate the cardinality of the exact Pareto front $|\mathcal{S}^*|$, of the approximated Pareto front $|\mathcal{S}|$ and of their intersection. We then combine these measures to obtain:

- the percentage of strict non-dominated solutions generated by NSGA-II:

$$\%Sol = \frac{|\mathcal{S} \cap \mathcal{S}^*|}{|\mathcal{S}|} \times 100\%$$

- the percentage of exact non-dominated solutions generated by NSGA-II:

$$\%exact = \frac{|\mathcal{S} \cap \mathcal{S}^*|}{|\mathcal{S}^*|} \times 100\%$$

- the percentage of weak non-dominated solutions generated by NSGA-II (compared to the exact Pareto front).

$$\%wSol = \frac{\sum_{b \in \mathcal{S}} g(\mathcal{S}^*, b)}{|\mathcal{S}|} \times 100\%$$

with $g(\mathcal{S}^*, b) =$

$$\begin{cases} 1 & \text{if } \exists a \in \mathcal{S}^* (f^A(b) = f^A(a) \wedge f^B(b) > f^B(a)) \\ & \vee (f^A(b) > f^A(a) \wedge f^B(b) = f^B(a)) \\ 0 & \text{otherwise} \end{cases}$$

Distances We calculate distances between the two fronts.

- The measure of Czyzak and Jaskiewicz (1998) is used. This measure is appropriate for our situation since it requires a reference set (which in our case is the exact Pareto front $\mathcal{S}^*$) and allows comparisons in terms of closeness to the reference set Balasubramanian et al. (2009). We define

$$c(a, b) = \max_{X \in \{A, B\}} \left\{ \frac{1}{w^X} (f^X(b) - f^X(a)) \right\}$$

and

$$Dist = \frac{1}{|\mathcal{S}^*|} \sum_{a \in \mathcal{S}^*} \min_{b \in \mathcal{S}} \{c(a, b)\}$$

Thus, if solutions a and b are identical, then $c(a, b) = 0$; otherwise, $c(a, b)$ is equal to the criterion that has the maximum weighted deviation. Here, w^X is the difference between the maximum and minimum values of the criterion f^X , $X \in \{A, B\}$. It represents the average maximum deviation between two Pareto fronts.

- We also use the generational distance: $GD = \frac{1}{|\mathcal{S}|} \left(\sum_{i=1}^{|\mathcal{S}|} d_i \right)$, with d_i the minimum Euclidean distance between the element $b_i \in \mathcal{S}$ and some element of $\mathcal{S}^*$. It measures an average distance between the two fronts.

Hypervolume Even if the convex hull of the Pareto front $\mathcal{S}$ is near to the convex hull of the Pareto front $\mathcal{S}^*$, an average distance may be a poor indicator of the quality of the front $\mathcal{S}$. Therefore, we used the metric $\mathcal{H}$ for hypervolume which calculates the area dominated by some front. In the following, we give the difference between the two normalized hypervolumes: the smaller this value, the better the quality of the approximated Pareto front.

6.3.2 Fixing the parameters

A subset of ten instances has been used for fixing the parameters of NSGA-II. The tests show that the best results are obtained when $N_{P0} = 50$ individuals, $N_{Pk} = 20$, the probability of mutation ρ is fixed to 0.05, and the number of iterations is fixed to 100.

6.4 Numerical results

The results for each problem are presented in the following tables.

- Table 7 (respectively, 8) presents the results for problem $Pm|IN|C_{\max}^A, C_{\max}^B$ where $n_B = \lfloor n_A/2 \rfloor$ (respectively, $n_B > \lfloor n_A/2 \rfloor$);

- Table 9 (respectively, 10) presents the results for problem $Pm|IN|C_{\max}^A, \sum C_j^B$ where $n_B = \lfloor n_A/2 \rfloor$ (respectively, $n_B > \lfloor n_A/2 \rfloor$);
- Table 11 presents the results for problem $Pm|IN|\sum C_j^A, \sum C_j^B$;
- Table 12 presents the results for problem $Pm|IN|\sum C_j^A, C_{\max}^B$.

In addition to the previous performance measures, we also computed the average computation time in seconds required to obtain the Pareto front for each method/model; it is denoted by CPU .

6.5 Discussion of the results

6.5.1 Note on the cardinality

For problems $\mathcal{P}(C^B S^A)$ and $\mathcal{P}(S^B S^A)$, the number of Pareto solutions increases with an increasing number of jobs, while the number of Pareto solutions for the $Pm|IN|C_{\max}^A, f^B$ problems is rather constant and varies between 1 and 3 when $n_B = \lfloor n_A/2 \rfloor$. However, this is not the case for the instances with $n_B > \lfloor n_A/2 \rfloor$. For example, 30 Pareto solutions are found for problem $\mathcal{P}(C^B C^A)$ with 2 machines, 20 job and $n_B = n_A - 1$. This is consistent with the observations in section 6.2.

Using linear combination approach (see Sect. 4.2), the MILP-Assign model generates a subset of the supported solutions (solutions on the hull curve) for the problem $\mathcal{P}(S^B S^A)$. In our work, we take $\alpha \in \{0, 0.1, 0.2, \dots, 0.9, 1.0\}$. The results are presented in Table 11. On average, this technique leads to less than 10 Pareto solutions, since it may happen that the method obtains the same points for two different values of α .

6.5.2 Note on the execution time

The proposed mathematical models present different performances. The time-indexed model reached a limit of one hour CPU time with $n_A = 20$ or 30 for each problem. Moreover, it is always outperformed by the MILP-Assign model. This is because the time-indexed model involves a pseudo-polynomial number of binary variables. The NSGA-II method is instantaneous, whatever the size of the Pareto front or the size of the instance.

6.5.3 Note on the distances

According to the tables, it can be concluded that the distances from the reference set are small for all of the problems.

- For problem $\mathcal{P}(C^B C^A)$, and when $n_B = \lfloor n_A/2 \rfloor$, on average 58% of the approximate Pareto solutions generated by the NSGA-II are exact solutions, which represents on average 56% of the whole exact Pareto front, while 35.5% of near Pareto solutions are strict weakly Pareto. The generational distance on average equals to 2.03. Since generational distance is calculated without normalization, we give in Table 13 the mean values of the optimized criteria. Measurements show that measured distances are very small compared to values. Indeed, the maximum distance recorded is equal to 5.81 which corresponds on average to 2.3% of the value of the global criterion. The minimum distance is equal to 0.16 which corresponds on average to 0.013% to the value of the global criterion.
- Concerning problem $\mathcal{P}(S^B C^A)$, and when $n_B = \lfloor n_A/2 \rfloor$, on average 85% of the approximate Pareto solutions generated by the NSGA-II are exact solutions, which represents on average 83% of the whole exact Pareto front, while 14% of near Pareto solutions are strict weakly Pareto. The generational distance on average equals to 1.74. Because this distance is not normalized, we conclude that the near Pareto fronts found by the NSGA-II are close to the optimal Pareto fronts.
- For $\mathcal{P}(S^B S^A)$, on average, 62% of the approximate Pareto solutions generated by the NSGA-II are exact solutions, and 31.8% solutions are strict weak Pareto solutions. On average, 31% of the whole exact Pareto front is found by NSGA-II. Although it is less than for problems minimizing $C_{\max}^A$, those solutions exhibit very good dispersion, as we will see in the following subsection 6.5.4 about hypervolume. Moreover, NSGA-II solutions are still close to the exact Pareto front. Indeed, the maximum generational distance recorded is equal to 6.97 which corresponds on average to 0.3% of the value of the global criterion according to Table 14. The minimum distance is equal to 1.82 which corresponds on average to 0.25% to the value of the global criterion.
- For $\mathcal{P}(C^B S^A)$, on average, 60% of the approximate Pareto solutions generated by the NSGA-II are exact solutions, and 39% solutions are strict weak Pareto solutions. On average, 36.7% of the whole exact Pareto front is found by NSGA-II, but it is a representative subset as shown by the hypervolume metric. The generational distance on average equals to 4.78. As already stated, since this distance is not normalized, we conclude that the near Pareto fronts found by the NSGA-II are close to the optimal Pareto fronts.

6.5.4 Note on the hypervolume

The hypervolume was calculated for the problems $\mathcal{P}(S^B S^A)$ and $\mathcal{P}(C^B S^A)$, the others admit a few number of Pareto solu-

tions, so this measure is not meaningful. These results can be interpreted as follows: for example, if $\mathcal{H} = 0.015$, then on average 1.5% of the area dominated by the exact Pareto front is not dominated by the near Pareto front. According to Tables 11 and 12, we can conclude that the approximate Pareto fronts are close to the exact Pareto fronts. Indeed, for $\mathcal{P}(S^B S^A)$ (resp. $\mathcal{P}(C^B S^A)$) the results show that on average 0.32% (resp. 0.47%) of the area dominated by the exact Pareto solutions is not dominated by the approximate Pareto solutions. Here, the hypervolume is a good indicator of the dispersion of the approximate solutions. Although NSGA-II generates less solutions than the MILP, it nearly covers the optimal area. Figure 11 illustrates this behavior on an instance with 2 machines and 15 jobs for the $\mathcal{P}(S^B S^A)$. The MILP-Time found 67 solutions, while NSGA-II provided 12 solutions. Still, these solutions are well distributed near the optimal front.

7 Conclusion

In this paper, we tackled the scheduling problems where two interfering agents compete to perform their jobs on common identical parallel machines. One global agent is associated with the whole set of jobs while the other is associated with a subset of jobs; finally, the agent job sets are non-disjoint. Each agent needs to minimize either the makespan criteria or the total completion time criteria. These objective functions depend only on the jobs of the concerned agent. Different combinations of the cost functions lead to four NP-hard problems. In fact, even when the agents aim at minimizing the same criteria, the problem remains difficult. In our work, we seek to generate the set of Pareto solutions for each problem.

As a first step, we proved that the problem minimizing makespan criteria for both agents admits a pseudopolynomial number of Pareto solutions. We also proposed a generic NSGA-II with an adequate encoding scheme for each problem. The heuristic approaches are compared with two types of integer programming formulations: time-indexed formulation and machine assignment formulations. The empirical results indicate that the heuristic approaches provide reasonable solution qualities while also being computationally efficient. This research also shows that exploiting problem structure in the different aspects of the genetic algorithm can produce good results.

The proposed genetic algorithm and mathematical programming formulations can be generalized to larger problems, with potentially other criteria. It can be also worthwhile to determine how the introduction of more agents affects the properties of the problems.

Acknowledgements The authors wish to thank the CaSciModOT structure (Calcul Scientifique et Modélisation Orléans-Tours), part of the

French national network of complex systems (RNSC - Réseau National des Systèmes Complexes), thanks to which the numerical experiments were conducted.

References

- Agnetis, A., Mirchandani, P., Pacciarelli, D., & Pacifici, A. (2000). Nondominated schedules for a job-shop with two competing users. *Computational & Mathematical Organization Theory*, 6, 191–217. <https://doi.org/10.1023/A:1009637419820>
- Agnetis, A., Mirchandani, P., Pacciarelli, D., & Pacifici, A. (2004). Scheduling problems with two competing agents. *Operations Research*, 52, 229–242. <https://doi.org/10.1287/opre.1030.0092>
- Agnetis, A., Billaut, J.-C., Gawiejnowicz, S., Pacciarelli, D., & Soukhal, A. (2014). *Multiagent scheduling. Models and algorithms*. <https://doi.org/10.1007/978-3-642-41880-8>
- Baker, K., & Smith, J. (2003). A multiple-criterion model for machine scheduling. *J. Scheduling*, 6, 7–16. <https://doi.org/10.1023/A:1022231419049>
- Balasubramanian, H., Fowler, J., Keha, A., & Pfund, M. (2009). Scheduling interfering job sets on parallel machines. *European Journal of Operational Research*, 199(1), 55–67. <https://doi.org/10.1016/j.ejor.2008.10.038>
- Belaid, R., Tkindt, V., & Esswein, C. (2012). Scheduling batches in flowshop with limited buffers in the shampoo industry. *Eur. J. Oper. Res.*, 223(2), 560–572. <https://doi.org/10.1016/j.ejor.2012.06.035>
- Bruno, J. L., Coffman, E. G., & Sethi, R. (1974). Scheduling independent tasks to reduce mean finishing time. *Commun. ACM*, 17, 382–387.
- Choi, B.-C., & Park, M.-J. (2017). Two-agent parallel machine scheduling with a restricted number of overlapped reserved tasks. *European Journal of Operational Research*, 260(2), 514–519. <https://doi.org/10.1016/j.ejor.2017.01.010>
- Cordeiro, D., Dutot, P.-F., Mounié, G., & Trystram, D. (2011). Tight analysis of relaxed multi-organization scheduling algorithms. *Proceedings - 25th IEEE International Parallel and Distributed Processing Symposium, IPDPS 2011*, <https://doi.org/10.1109/IPDPS.2011.112>.
- Czyzak, P., & Jaskiewicz, A. (1998). Pareto simulated annealing—a metaheuristic technique for multiple-objective combinatorial optimization. *Journal of Multi-Criteria Decision Analysis*, 7(1), 34–47.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: Nsga-ii. *Evolutionary Computation, IEEE Transactions on*, 6, 182–197. <https://doi.org/10.1109/4235.996017>
- D'Souza, R., Sekaran, C., & Kandasamy, A. (2010). Improved nsga-ii based on a novel ranking scheme. *Journal of Computing*, 2, 02.
- Elvikis, D., & T'Kindt, V. (2014). Two-agent scheduling on uniform parallel machines with maximum criteria functions. *Annals of Operations Research*, 213, 79–94. <https://doi.org/10.1007/s10479-012-1099-0>. <https://hal.archives-ouvertes.fr/hal-01004005>.
- Elvikis, D., Hamacher, H., & T'Kindt, V. (2009). Scheduling two interfering job sets on uniform parallel machines with sum and makespan criteria. In *9th Workshop on Models and Algorithms for Planning and Scheduling Problems*, pages –, Rolduc, Netherlands. <https://hal.archives-ouvertes.fr/hal-01025649>.
- Gerstl, E., & Mosheiov, G. (2013). Scheduling problems with two competing agents to minimized weighted earliness-tardiness. *Computers & Operations Research*, 40(1), 109–116. <https://doi.org/10.1016/j.cor.2012.05.019>

- Gu, M., Gu, J., & Lu, X. (2018). An algorithm for multi-agent scheduling to minimize the makespan on m parallel machines. *Journal of Scheduling*, 21, 10. <https://doi.org/10.1007/s10951-017-0546-9>
- Gu, M., Lu, X., & Gu, J. (2019). An approximation algorithm for multi-agent scheduling on two uniform parallel machines. *Optimization Letters*, 13, 06. <https://doi.org/10.1007/s11590-018-1298-y>
- Hosni, E., Kolsi, N., Chaari, W. L., & Ghédira, K. (2022). Efficient resource allocation using a multi-criteria approach and nodes clustering for heterogeneous hadoop cluster. In M. Cristani, C. Toro, C. Zanni-Merk, R. J. Howlett, and L. C. Jain, editors, *Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 26th International Conference KES-2022, Verona, Italy and Virtual Event, 7-9 September 2022*, volume 207 of *Procedia Computer Science*, pages 2794–2803. Elsevier. <https://doi.org/10.1016/j.procs.2022.09.337>
- Huynh Tuong, N., Soukhal, A., & Billaut, J.-C. (2012). Single-machine multi-agent scheduling problems with a global objective function. *Journal of Scheduling*, 15, 311–321. <https://doi.org/10.1007/s10951-011-0252-y>
- Jans, R. (2009). Solving lot-sizing problems on parallel identical machines using symmetry-breaking constraints. *INFORMS Journal on Computing*, 21, 123–136. <https://doi.org/10.1287/ijoc.1080.0283>
- Konak, A., Coit, D. W., & Smith, A. E. (2006). Multi-objective optimization using genetic algorithms: A tutorial. *Reliability Engineering & System Safety*, 91(9), 992–1007. <https://doi.org/10.1016/j.res.2005.11.018>
- Kovalyov, M. Y., Ng, C., & Cheng, T. E. (2007). Fixed interval scheduling: Models, applications, computational complexity and algorithms. *European Journal of Operational Research*, 178(2), 331–342. <https://doi.org/10.1016/j.ejor.2006.01.049>
- Kung, H., Luccio, F., & Preparata, F. (1975). On finding the maxima of a set of vectors. *Journal of the ACM (JACM)*, 22, 469–476. <https://doi.org/10.1145/321906.321910>
- Lee, W.-C., Wang, J.-Y., & Lin, M.-C. (2016). A branch-and-bound algorithm for minimizing the total weighted completion time on parallel identical machines with two competing agents. *Knowledge-Based Systems*, 105, 68–82. <https://doi.org/10.1016/j.knosys.2016.05.012>
- Leung, J., Pinedo, M., & Wan, G. (2010). Competitive two-agent scheduling and its applications. *Operations Research*, 58(2), 458–469. <https://doi.org/10.1287/opre.1090.0744>
- Li, D., & Lu, X. (2017). Two-agent parallel-machine scheduling with rejection. *Theoretical Computer Science*, 703, 66–75. <https://doi.org/10.1016/j.tcs.2017.09.004>
- Liu, C.-H., Nanthapodej, R., & Hsu, S.-Y. (2018). Scheduling two interfering job sets on parallel machines under peak power constraint. *Production Engineering*, 12, 06. <https://doi.org/10.1007/s11740-018-0840-1>
- Olariu, S. (1991). An optimal greedy heuristic to color interval graphs. *Information Processing Letters*, 37(1), 21–25. [https://doi.org/10.1016/0020-0190\(91\)90245-D](https://doi.org/10.1016/0020-0190(91)90245-D)
- Pei, J., Wei, J., Liao, B., Liu, X., & Pardalos, P. M. (2020). Two-agent scheduling on bounded parallel-batching machines with an aging effect of job-position-dependent. *Annals of Operations Research*, 294(1), 191–223. https://EconPapers.repec.org/RePEc:spr:annopr:v:294:y:2020:i:1:d:10.1007_s10479-019-03160-y
- Sadi, F., & Soukhal, A. (2017). Complexity analyses for multi-agent scheduling problems with a global agent and equal length jobs. *Discrete Optimization*, 23, 93–104. <https://doi.org/10.1016/j.disopt.2017.01.001>. <https://hal.archives-ouvertes.fr/hal-03063865>
- Sadi, F., Soukhal, A., & Billaut, J.-C. (2014). Solving multi-agent scheduling problems on parallel machines with a global objective function. *RAIRO-Oper. Res.*, 48(2), 255–269. <https://doi.org/10.1051/ro/2014005>
- Smith, B., & Dh, H. (2001). Reducing symmetry in a combinatorial design problem. *Proceedings of CPAIOR'01*, 351–359.
- T'kindt, V., & Billaut, J.-C. (2006). *Multicriteria Scheduling: Theory, Models and Algorithms*.
- Wan, G., Vakati, S. R., Leung, J. Y.-T., & Pinedo, M. (2010). Scheduling two agents with controllable processing times. *European Journal of Operational Research*, 205(3), 528–539. <https://doi.org/10.1016/j.ejor.2010.01.005>
- Yin, Y., Cheng, S., Cheng, E., Wang, D., & Wu, C. (2016). Just-in-time scheduling with two competing agents on unrelated parallel machines. *Omega*, 63, 41–47. <https://doi.org/10.1016/j.omega.2015.09.010>
- Yin, Y., Chen, Y., Qin, K., & Wang, D. (2019). Two-agent scheduling on unrelated parallel machines with total completion time and weighted number of tardy jobs criteria. *Journal of Scheduling*, 22, 06. <https://doi.org/10.1007/s10951-018-0583-z>
- Yu, F., Wen, P., & Yi, S. (2018). A multi-agent scheduling problem for two identical parallel machines to minimize total tardiness time and makespan. *Advances in Mechanical Engineering*, 10, 168781401875610. <https://doi.org/10.1177/1687814018756103>
- Zhao, K., & Lu, X. (2013). Approximation schemes for two-agent scheduling on parallel machines. *Theoretical Computer Science*, 468, 114–121. <https://doi.org/10.1016/j.tcs.2012.11.002>
- Zhao, K., & Lu, X. (2016). Two approximation algorithms for two-agent scheduling on parallel machines to minimize makespan. *Journal of Combinatorial Optimization*, 31(1), 260–278. <https://doi.org/10.1007/s10878-014-9744-y>
- Zhao, K., Lu, X., & Gu, M. (2016). A new approximation algorithm for multi-agent scheduling to minimize makespan on two machines. *Journal of Scheduling*, 19, 21–31.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.